

# Le désabonnement des clients : Un problème de classification à résoudre avec Python

## Un guide pour prédire le désabonnement des clients d'un service de cartes de crédit

*L'attrition de la clientèle est une préoccupation majeure pour les entreprises de divers secteurs, en particulier pour les banques et les institutions financières. Dans le monde des services de cartes de crédit, la fidélisation des clients est cruciale car l'acquisition de nouveaux clients est souvent plus coûteuse que la fidélisation des clients existants.*

**Les responsables d'une banque ont remarqué qu'un nombre croissant de clients abandonnaient leurs services de cartes de crédit. Cette augmentation inattendue du taux de désabonnement n'est pas seulement une perte de revenus, c'est aussi le signe que quelque chose doit changer. Le directeur est impatient d'identifier à l'avance les clients potentiels afin que la banque puisse s'engager de manière proactive avec eux, comprendre leurs préoccupations et leur offrir des services améliorés pour les fidéliser.**

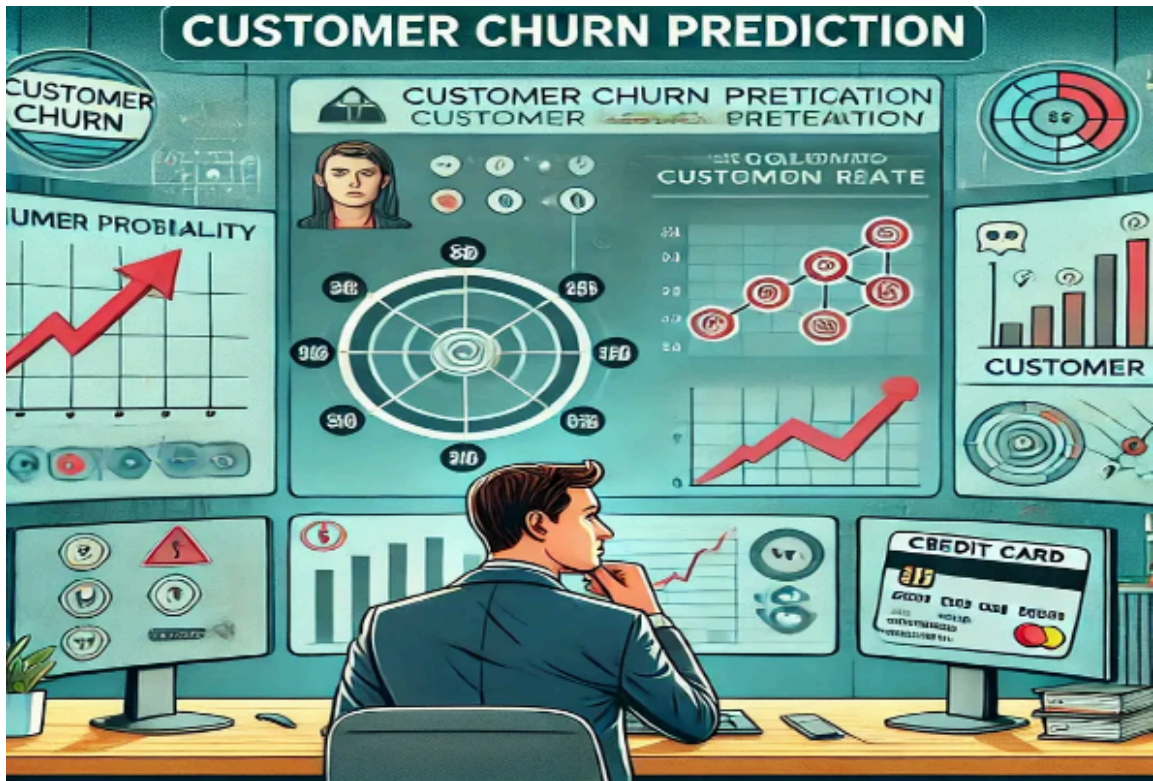
Pour résoudre ce problème, nous avons pris la responsabilité d'effectuer une analyse approfondie afin d'aider la banque à identifier les clients susceptibles de changer de fournisseur.

En tirant parti de l'analyse prédictive, nous cherchons à identifier des modèles dans le comportement des clients, les détails démographiques et les attributs financiers qui peuvent signaler une plus grande probabilité de désabonnement.

Pour ce faire, nous utiliserons l'ensemble de données Credit Card Customers disponible sur Kaggle, qui contient les données de 10 000 clients avec environ 18 caractéristiques différentes telles que l'âge, le salaire, l'état civil, la limite de la carte de crédit, la catégorie de la carte de crédit, et plus encore.

L'objectif de cette analyse n'est pas seulement de construire un modèle de prédiction du taux d'attrition, mais aussi de mieux comprendre les facteurs qui influencent l'attrition de la clientèle. En comprenant ces facteurs, la banque peut formuler des stratégies efficaces pour réduire les taux de désabonnement et améliorer la satisfaction des clients.

Dans cet article, nous allons parcourir une approche étape par étape pour prédire le taux d'attrition des clients à l'aide de divers algorithmes d'apprentissage automatique, ainsi que l'analyse des caractéristiques clés qui jouent un rôle fondamental pour déterminer si un client va rester ou partir.



## INTRODUCTION

La perte de clientèle est une préoccupation majeure pour les entreprises de divers secteurs, en particulier pour les banques et les institutions financières. Dans le monde des services de cartes de crédit, la fidélisation des clients est cruciale, car l'acquisition de nouveaux clients est souvent plus coûteuse que la fidélisation des clients existants.

## CONTEXT

Un responsable d'une banque a remarqué qu'un nombre croissant de clients abandonnaient leurs services de carte de crédit. Cette augmentation inattendue du taux de désabonnement ne constitue pas seulement une perte de revenus, mais aussi un signal qu'il faut changer les choses. Le responsable souhaite identifier à l'avance les clients potentiels qui abandonnent leur compte afin que la banque puisse s'engager de manière proactive auprès d'eux, comprendre leurs préoccupations et leur proposer des services améliorés pour les fidéliser.

Pour résoudre ce problème, nous avons pris la responsabilité de réaliser une analyse approfondie pour aider la banque à identifier les clients susceptibles de quitter son compte.

En tirant parti de l'analyse prédictive, nous cherchons à identifier des modèles de comportement des clients, des détails démographiques et des attributs financiers qui peuvent signaler une probabilité plus élevée de désabonnement.

À cette fin, nous utiliserons l'ensemble de données des clients de cartes de crédit disponible sur Kaggle, qui contient des données pour 10 000 clients avec environ 18 caractéristiques différentes telles que l'âge, le salaire, l'état matrimonial, la limite de la carte de crédit, la catégorie de carte de crédit, etc.

Ce guide conceptuel vous fournira une solide compréhension des étapes impliquées dans l'analyse du taux de désabonnement des clients à l'aide de diverses techniques d'apprentissage automatique. Nous aborderons le prétraitement des données, la sélection des fonctionnalités, la création de modèles et l'évaluation de plusieurs algorithmes pour prédire efficacement le taux de désabonnement des clients. Pour l'implémentation détaillée du code, vous pouvez vous référer au bloc-notes Kaggle. À la fin de cet article, vous aurez une idée plus claire de la manière d'utiliser les méthodes de science des données pour identifier les clients potentiels et aider la banque à fidéliser ses précieux clients.

## 1. Introduction à l'analyse du taux de désabonnement des clients

Ce carnet a pour objectif d'explorer l'utilisation des techniques d'apprentissage automatique pour prédire efficacement le taux de désabonnement des clients. En analysant les données clients, nous pouvons identifier les modèles et les facteurs qui contribuent au taux de désabonnement, ce qui permet à la banque de s'engager de manière proactive auprès des clients à risque et d'améliorer ses stratégies de fidélisation.

Nous utiliserons l'ensemble de données clients de cartes de crédit disponible sur Kaggle, qui contient des informations détaillées sur la démographie des clients, les attributs du compte et l'historique des transactions.

# 1. Introduction to Customer Churn Analysis

Dans cet exercice nous allons utiliser des techniques d'apprentissage automatique pour prédire efficacement l'attrition de la clientèle. En analysant les

données des clients, nous pouvons identifier les modèles et les facteurs qui contribuent au désabonnement, ce qui permet à la banque de s'engager de manière proactive avec les clients à risque et d'améliorer les stratégies de fidélisation.

Nous utiliserons le jeu de données Credit Card Customer disponible sur Kaggle, qui contient des informations détaillées sur les caractéristiques démographiques des clients, les attributs des comptes et l'historique des transactions.

Ce carnet a pour objectif d'explorer l'utilisation des techniques d'apprentissage automatique pour prédire efficacement le taux de désabonnement des clients. En analysant les données clients, nous pouvons identifier les modèles et les facteurs qui contribuent au taux de désabonnement, ce qui permet à la banque de s'engager de manière proactive auprès des clients à risque et d'améliorer ses stratégies de fidélisation.

Nous utiliserons l'ensemble de données clients de cartes de crédit disponible sur Kaggle, qui contient des informations détaillées sur la démographie des clients, les attributs du compte et l'historique des transactions.

## Libraries and utilities

### 1.1 Importer des bibliothèques

La première étape de tout projet d'apprentissage automatique consiste à importer les bibliothèques nécessaires. Ces bibliothèques fournissent des outils pour la manipulation des données, la visualisation, la construction de modèles et l'évaluation.

```
In [14]: import numpy as np # Linear algebra
import pandas as pd # Data processing
from numpy import log
from datetime import date
import seaborn as sns # Data visualization
import matplotlib.ticker as mtick # Formatting axes in plots
import matplotlib.pyplot as plt
import plotly.graph_objects as go # Interactive plots

from sklearn.preprocessing import LabelEncoder, OneHotEncoder, MinMaxScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split, cross_val_score, RepeatedCrossValidator
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from lightgbm import LGBMClassifier

from sklearn.metrics import confusion_matrix, precision_score
from sklearn.model_selection import GridSearchCV, StratifiedKFold

from imblearn.pipeline import Pipeline as imbpipeline
from imblearn.over_sampling import SMOTE # Synthetic Minority Over-sampling

from scipy.stats import randint as sp_randint, uniform as sp_uniform

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
sns.set(style='white')

```

Ces bibliothèques incluent des outils de manipulation de données ( numpy, pandas), de visualisation ( seaborn, matplotlib, plotly) et d'apprentissage automatique ( scikit-learn, lightgbm, imbalanced-learn). Elles fournissent les fonctionnalités nécessaires pour prétraiter les données, créer et évaluer des modèles et gérer le déséquilibre des classes à l'aide de techniques telles que SMOTE (nous verrons les détails plus tard).

## Librairies de Base

### 1. NumPy ( import numpy as np )

- **Description:** Bibliothèque fondamentale pour le calcul scientifique en Python. Elle fournit des structures de données puissantes, comme les tableaux multidimensionnels (arrays), ainsi que des fonctions mathématiques pour effectuer des opérations rapides et efficaces sur ces tableaux.
- **Utilisation:** Manipulation de données numériques, calculs algébriques, transformations mathématiques.

### 2. Pandas ( import pandas as pd )

- **Description:** Librairie essentielle pour la manipulation et l'analyse des données. Elle offre des structures de données flexibles comme les DataFrames, qui facilitent le nettoyage, la transformation et l'analyse des données structurées.
- **Utilisation:** Gestion des données, importation/exportation de fichiers (CSV, Excel, etc.), manipulation de DataFrames.

### 3. Logarithme de NumPy ( from numpy import log )

- **Description:** Fonction spécifique de NumPy pour calculer le logarithme naturel.

- **Utilisation:** Transformation des données, calculs statistiques.

#### 4. **DateTime ( from datetime import date )**

- **Description:** Module intégré de Python pour manipuler les dates et les heures.
- **Utilisation:** Gestion des dates, calculs de durées, formatage des dates.

## Visualisation des Données

#### 5. **Seaborn ( import seaborn as sns )**

- **Description:** Bibliothèque de visualisation basée sur Matplotlib qui simplifie la création de graphiques statistiques attrayants.
- **Utilisation:** Création de graphiques comme les heatmaps, les boxplots, les distributions, etc.

#### 6. **Matplotlib Ticker ( import matplotlib.ticker as mtick )**

- **Description:** Module de Matplotlib pour la mise en forme des axes des graphiques.
- **Utilisation:** Personnalisation des échelles, des formats de tick marks, etc.

#### 7. **Matplotlib Pyplot ( import matplotlib.pyplot as plt )**

- **Description:** Interface de haut niveau de Matplotlib pour créer des visualisations statiques.
- **Utilisation:** Création de graphiques simples comme les lignes, les barres, les scatter plots, etc.

#### 8. **Plotly Graph Objects ( import plotly.graph\_objects as go )**

- **Description:** Module de Plotly pour créer des graphiques interactifs et dynamiques.
- **Utilisation:** Visualisations interactives pour le web, tableaux de bord, graphiques 3D.

## Prétraitement des Données avec scikit-learn

#### 9. **LabelEncoder, OneHotEncoder, MinMaxScaler ( from sklearn.preprocessing import LabelEncoder, OneHotEncoder, MinMaxScaler )**

- **Description:**
  - **LabelEncoder:** Encode les étiquettes non numériques en nombres.
  - **OneHotEncoder:** Convertit les variables catégorielles en une représentation binaire.
  - **MinMaxScaler:** Normalise les données en les mettant à l'échelle entre une plage spécifiée (généralement 0 et 1).

- **Utilisation:** Prétraitement des données pour les modèles de machine learning, gestion des variables catégorielles et numériques.

## Imputation et Sélection de Modèles

### 10. **SimpleImputer** ( `from sklearn.impute import SimpleImputer` )

- **Description:** Permet de gérer les valeurs manquantes dans les données en les imputant selon différentes stratégies (moyenne, médiane, mode, etc.).
- **Utilisation:** Nettoyage des données avant l'entraînement des modèles.

### 11. **Train Test Split, Cross Val Score, Repeated Stratified K-Fold** ( `from sklearn.model_selection import train_test_split, cross_val_score, RepeatedStratifiedKFold` )

- **Description:**
  - **train\_test\_split:** Divise les données en ensembles d'entraînement et de test.
  - **cross\_val\_score:** Évalue les performances d'un modèle à l'aide de la validation croisée.
  - **RepeatedStratifiedKFold:** Variante de la validation croisée qui répète la stratification des folds pour une évaluation plus robuste.
- **Utilisation:** Évaluation et validation des modèles de machine learning.

## Pipelines et Transformations

### 12. **Pipeline** ( `from sklearn.pipeline import Pipeline` )

- **Description:** Permet de chaîner plusieurs étapes de prétraitement et de modélisation en une seule séquence.
- **Utilisation:** Simplification du flux de travail, reproduction des transformations.

### 13. **ColumnTransformer** ( `from sklearn.compose import ColumnTransformer` )

- **Description:** Applique différentes transformations à différentes colonnes d'un DataFrame.
- **Utilisation:** Gestion des prétraitements spécifiques à certaines caractéristiques.

## Modèles de Classification

### 14. **DecisionTreeClassifier** ( `from sklearn.tree import DecisionTreeClassifier` )

- **Description:** Algorithme d'arbre de décision pour la classification.
- **Utilisation:** Modélisation de décisions basées sur des règles hiérarchiques.

#### 15. **SVC ( from sklearn.svm import SVC )**

- **Description:** Support Vector Classifier, une implémentation des machines à vecteurs de support.
- **Utilisation:** Classification binaire ou multi-classe avec une marge maximale.

#### 16. **Ensembles de Classificateurs ( from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, BaggingClassifier )**

- **Description:**
  - **RandomForestClassifier:** Ensemble d'arbres de décision pour améliorer la précision et réduire le surapprentissage.
  - **GradientBoostingClassifier:** Améliore les performances en combinant plusieurs modèles faibles de manière séquentielle.
  - **BaggingClassifier:** Méthode d'ensemble qui combine plusieurs modèles pour réduire la variance.
- **Utilisation:** Amélioration des performances de classification grâce aux techniques d'ensemble.

#### 17. **LGBMClassifier ( from lightgbm import LGBMClassifier )**

- **Description:** Implémentation de Gradient Boosting optimisée pour la vitesse et la performance, particulièrement efficace avec de grands ensembles de données.
- **Utilisation:** Classification rapide et précise, souvent utilisé dans les compétitions de machine learning.

## Évaluation et Optimisation des Modèles

#### 18. **Confusion Matrix, Precision Score ( from sklearn.metrics import confusion\_matrix, precision\_score )**

- **Description:**
  - **confusion\_matrix:** Matrice de confusion pour évaluer les performances d'un modèle de classification.
  - **precision\_score:** Mesure la précision des prédictions positives.
- **Utilisation:** Évaluation des performances des modèles de classification.

#### 19. **GridSearchCV, StratifiedKFold ( from sklearn.model\_selection import GridSearchCV, StratifiedKFold )**

- **Description:**

- **GridSearchCV**: Recherche des meilleurs hyperparamètres en utilisant la validation croisée.
- **StratifiedKFold**: Validation croisée qui conserve la distribution des classes dans chaque fold.
- **Utilisation**: Optimisation des hyperparamètres des modèles.

## Traitement des Données Déséquilibrées

20. **Pipeline d'Imbalanced-learn, SMOTE** ( `from imblearn.pipeline import Pipeline as imbpipeline, from imblearn.over_sampling import SMOTE` )

- **Description**:
  - **imbpipeline**: Pipeline spécifique à imbalanced-learn pour gérer les données déséquilibrées.
  - **SMOTE**: Technique de sur-échantillonnage synthétique des classes minoritaires.
- **Utilisation**: Traitement des jeux de données déséquilibrés pour améliorer les performances des modèles.

## Statistiques et Optimisation

21. **Randint et Uniform de SciPy** ( `from scipy.stats import randint as sp_randint, uniform as sp_uniform` )

- **Description**:
  - **randint**: Génère des entiers aléatoires dans une plage spécifiée.
  - **uniform**: Génère des nombres réels aléatoires uniformément distribués.
- **Utilisation**: Distribution des hyperparamètres pour l'optimisation, simulations statistiques.

## Gestion des Avertissements et Style

22. **Warnings** ( `import warnings` )

- **Description**: Module pour gérer les avertissements en Python.
- **Utilisation**: Contrôle de l'affichage des avertissements, comme ignorer les `FutureWarning`.

23. **Configuration de Seaborn** ( `sns.set(style='white')` )

- **Description**: Configure le style des graphiques Seaborn.
- **Utilisation**: Amélioration esthétique des visualisations avec un fond blanc.

## Résumé des Importations

- **Bibliothèques de Base:** `numpy`, `pandas`, `datetime` pour la manipulation des données.
- **Visualisation:** `seaborn`, `matplotlib`, `plotly` pour créer des graphiques statiques et interactifs.
- **Prétraitement et Modélisation:** `scikit-learn` et `imblearn` pour le prétraitement des données, la création de pipelines, et l'entraînement de modèles de classification.
- **Évaluation et Optimisation:** Modules de `scikit-learn` pour évaluer et optimiser les modèles.
- **Statistiques:** `scipy` pour les distributions statistiques utilisées dans l'optimisation des hyperparamètres.
- **Gestion des Avertissements et Style:** `warnings` pour gérer les avertissements et configuration de l'esthétique des graphiques.

Ces bibliothèques forment un écosystème puissant pour le traitement des données, l'analyse exploratoire, la modélisation prédictive et l'évaluation des performances des modèles de machine learning. Elles sont couramment utilisées dans des projets de science des données et d'apprentissage automatique.

## Custom Functions

**Les fonctions personnalisées sont utilisées pour simplifier les tâches répétitives telles que la visualisation de données ou l'évaluation de modèles. Vous trouverez ci-dessous quelques fonctions personnalisées utilisées dans l'analyse :**

- **bar\_plot:** Trace une colonne spécifique d'un DataFrame sous forme de graphique à barres.
- **bar\_plot\_tot:** Similaire à `bar_plot`, mais utilisé pour tracer le nombre total d'une colonne spécifique.
- **bar\_stacked\_plot:** Trace un graphique à barres empilées en fonction du nombre d'agrégations de fonctionnalités/étiquettes.
- **dist\_plot\_kde:** Trace la distribution d'une fonctionnalité pour des valeurs d'étiquette positives ou négatives à l'aide de tracés KDE.
- **evaluate\_model:** Évalue un modèle d'apprentissage automatique à l'aide d'une validation croisée.
- **get\_models:** Renvoie une liste de modèles d'apprentissage automatique à tester.

In [ ]:

```

In [15]: def bar_plot(df,ax, column_name= None,xlabel= None, ylabel= None, title= None
        """
        Python function to plot a specific column of dataframe
        df: dataframe
        column_name: column to plot
        xlabel: x axis label
        ylabel: y axis label
        colors: colors of the bars
        """

        #colors
        colors = colors

        #percentage of total
        df = df[column_name].value_counts()*100.0 /len(df)
        #reorder
        if reorderlist is not None:
            df = df.reindex(reorderlist)

        #group by column and count
        df.plot(ax= ax,kind='bar',
                stacked = True,
                rot = 0,
                color = colors,
                #figsize = (15,8)
                )

        #format axis and labels
        #ax.yaxis.set_major_formatter(mtick.PercentFormatter())
        ax.yaxis.set_visible(False)
        ax.set_ylabel(ylabel,size = 14)
        ax.set_xlabel(xlabel,size = 14)
        ax.set_title(title, size = 14)

        ax.spines['top'].set_visible(False)
        ax.spines['right'].set_visible(False)
        ax.spines['bottom'].set_visible(False)
        ax.spines['left'].set_visible(False)

        # create a list to collect the plt.patches data
        totals = []

        # find the values and append to list
        for i in ax.patches:
            totals.append(i.get_width())

        # set individual bar labels using above list
        total = sum(totals)

        for i in ax.patches:
            # get_width pulls left or right; get_y pushes up or down
            ax.text(i.get_x()+.10, i.get_height()-8.0, str(round((i.get_height()/
                    fontsize=16,
                    color='white',
                    weight = 'bold',
                    )

```

```

    return

#####

def bar_plot_tot(df, ax, column_name= None, xlabel= None, ylabel= None, title
    """
    Python function to plot a specific column of dataframe
    df: dataframe
    column_name: column to plot
    xlabel: x axis label
    ylabel: y axis label
    colors: colors of the bars
    """

    #colors
    colors = colors
    #totald
    df = df[column_name].value_counts()
    #reorder
    if reorderlist is not None:
        df = df.reindex(reorderlist)

    #group by column and count
    df.plot.barh(ax= ax,
                  stacked = True,
                  rot = 0,
                  color = colors,
                  # figsize = (15,8)
                  )
    #format axis and labels
    # ax.yaxis.set_major_formatter(mtick.PercentFormatter())
    ax.set_ylabel(ylabel, size = 14)
    ax.set_xlabel(xlabel, size = 14)
    ax.set_title(title, size = 14)

    ax.xaxis.set_visible(False)

    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['bottom'].set_visible(False)
    ax.spines['left'].set_visible(True)

    # create a list to collect the plt.patches data
    totals = []

    # find the values and append to list
    for i in ax.patches:
        totals.append(i.get_width())
    for i, v in enumerate(totals):
        ax.text(v/2-(v/3), i, str(int(v)), color='white', fontweight='bold',

    return

#####

```

```

def bar_stacked_plot(df, ax, label=None, feature=None, title=None, colors=[
    """
    Python function to plot a stacked bar plot based on feature/label aggreg
    df: dataframe
    label : label of dataframe
    feature: feature of dataframe
    title: plot title
    colors: colors of the bars
    reorderlist: specify order if ordinal
    """

    # Group by label and feature and count the feature
    df = df.groupby([feature, label]).size().unstack()

    # Reorder if needed
    if reorderlist is not None:
        df = df.reindex(reorderlist)

    # Plot the stacked bar chart
    (df.T * 100.0 / df.T.sum()).T.plot(
        kind='bar',
        ax=ax,
        width=0.6,
        stacked=True,
        rot=0,
        color=colors
    )

    # Set font
    csfont = {'fontname': 'sans-serif'}

    # Set axis parameters, title, and labels
    ax.yaxis.set_visible(False)
    ax.set_ylabel('% ' + feature, size=14, **csfont)
    ax.set_xlabel('')
    ax.legend(loc='center left', bbox_to_anchor=(1, 0.5), prop={'size': 14},
    ax.set_title(title, size=14, **csfont)

    # Hide spines
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['bottom'].set_visible(False)
    ax.spines['left'].set_visible(False)

    # Add data labels at the center of each bar
    for p in ax.patches:
        width = p.get_width()
        height = p.get_height()
        x = p.get_x()
        y = p.get_y()

        # Center the label in the middle of the bar
        ax.annotate(
            '{:.0f}%'.format(height),
            (x + width / 2, y + height / 2),
            ha='center', va='center',

```

```

        color='white',
        weight='bold',
        size=14
    )

    return

#####

def dist_plot_kde(df, ax, label, feature ,xlabel= None, title= None):
    """
    Python function to plot the distribution of a feature for positive or ne
    df: dataframe
    ax: plot definition
    label: label
    feature: feature
    title: title of the plot
    xlabel: x axis label

    """
    #positive label
    ax = sns.kdeplot(cust_df[feature].loc[(cust_df[label] == 1)],
                    color="Red", shade = True)
    #negative label
    ax = sns.kdeplot(cust_df[feature].loc[(cust_df[label] == 0)],
                    ax =ax, color="Blue", shade= True, alpha= 0.2)

    #legend and other plot labels
    ax.legend([label,"Not " + label ],loc='upper right')
    ax.set_ylabel('Density')
    ax.set_xlabel(xlabel)
    ax.set_title(title)

    return

#####

# evaluate a model
def evaluate_model(X, y, model):
    # define evaluation procedure
    cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
    # evaluate model
    scores = cross_val_score(model, X, y, scoring='recall', cv=cv, n_jobs=-1)
    return scores

#####

# define models to test
def get_models():
    models, names = list(), list()
    # CART
    models.append(DecisionTreeClassifier())
    names.append('CART')
    # SVM

```

```

models.append(SVC(gamma='scale'))
names.append('SVM')
# Bagging
models.append(BaggingClassifier(n_estimators=100))
names.append('BAG')
# RF
models.append(RandomForestClassifier(n_estimators=100))
names.append('RF')
# GBM
models.append(GradientBoostingClassifier(n_estimators=100))
names.append('GBM')
#LightGBM
models.append(LGBMClassifier())
names.append('LightGBM')
# LR
# models.append(LogisticRegression())
# names.append('LR')
return models, names

```

Ces fonctions sont conçues pour rationaliser le processus de création de visualisations et d'évaluation des modèles, améliorant à la fois l'interprétabilité et l'efficacité de l'analyse.

## Load the Data

Pour commencer l'analyse, l'ensemble de données est chargé à partir d'un fichier CSV dans un DataFrame pandas. Les données contiennent divers attributs des clients, notamment des informations démographiques, l'historique des transactions et les détails du compte.

```

In [16]: # Load the dataset from CSV file
cust_df = pd.read_csv('BankChurners.csv')

```

```

In [17]: df = cust_df.copy()

df.head()

```

```
Out[17]:
```

|   | CLIENTNUM | Attrition_Flag    | Customer_Age | Gender | Dependent_count | Educ |
|---|-----------|-------------------|--------------|--------|-----------------|------|
| 0 | 768805383 | Existing Customer | 45           | M      | 3               |      |
| 1 | 818770008 | Existing Customer | 49           | F      | 5               |      |
| 2 | 713982108 | Existing Customer | 51           | M      | 3               |      |
| 3 | 769911858 | Existing Customer | 40           | F      | 4               |      |
| 4 | 709106358 | Existing Customer | 40           | M      | 3               |      |

5 rows × 23 columns

```
In [ ]:
```

L'ensemble de données, intitulé « Clients titulaires d'une carte de crédit », comprend 10 000 enregistrements et près de 18 caractéristiques. Il offre une vue complète des caractéristiques des clients, notamment l'âge, le revenu, le niveau d'éducation, l'état civil, la limite de la carte de crédit et le comportement des transactions . Ces données seront utilisées pour créer des modèles prédictifs de désabonnement des clients.

```
In [18]: cust_df.sample()
```

```
Out[18]:
```

|      | CLIENTNUM | Attrition_Flag    | Customer_Age | Gender | Dependent_count | E |
|------|-----------|-------------------|--------------|--------|-----------------|---|
| 7446 | 719463708 | Existing Customer | 47           | F      | 1               |   |

1 rows × 23 columns

```
In [19]: # Supprimer les deux dernières colonnes et la colonne du numéro de client,
cust_df = cust_df.iloc[:, :-2]
cust_df.drop(columns=["CLIENTNUM"], inplace=True)

#
```

```
In [20]: # Afficher les premières lignes du jeu de données cust_df.head()
cust_df.head()
```

| Out[20]: | Attrition_Flag    | Customer_Age | Gender | Dependent_count | Education_Level |
|----------|-------------------|--------------|--------|-----------------|-----------------|
| 0        | Existing Customer | 45           | M      | 3               | High School     |
| 1        | Existing Customer | 49           | F      | 5               | Graduate        |
| 2        | Existing Customer | 51           | M      | 3               | Graduate        |
| 3        | Existing Customer | 40           | F      | 4               | High School     |
| 4        | Existing Customer | 40           | M      | 3               | Uneducated      |

## Dataset Description

In [21]: `df.shape`

Out[21]: (10127, 23)

### 1.4 Dataset Description

- **Attrition\_Flag:** Indique si le compte est fermé (1) ou actif (0).
- **Customer\_Age:** Âge du client en années.
- **Gender:** Sexe du client (M/F).
- **Dependent\_count:** Nombre de personnes à charge associées au compte.
- **Education\_Level:** Niveau d'études du titulaire du compte..
- **Marital\_Status:** État civil du client (marié, célibataire, divorcé, etc.).
- **Income\_Category:** Catégorie de revenu annuel du titulaire du compte.
- **Card\_Category:** Type de carte de crédit détenue par le client (Blue, Silver, Gold, Platinum).
- **Months\_on\_book:** Période de la relation du client avec la banque.
- **Total\_Relationship\_Count:** Nombre total de produits détenus par le client..
- **Months\_Inactive\_12\_mon:** Nombre de mois pendant lesquels le compte a été inactif au cours des 12 derniers mois.

- **Contacts\_Count\_12\_mon:** Nombre de contacts établis au cours des 12 derniers mois.
- **Credit\_Limit:** Limite de crédit attribuée au client.
- **Total\_Revolving\_Bal:** Solde renouvelable de la carte de crédit.
- **Avg\_Open\_To\_Buy:** Ligne de crédit moyenne ouverte à l'achat (crédit disponible).
- **Total\_Amt\_Chng\_Q4\_Q1:** Variation du montant de la transaction entre Q4 et Q1.
- **Total\_Trans\_Amt:** Montant total des transactions au cours des 12 derniers mois..
- **Total\_Trans\_Ct:** Nombre total de transactions au cours des 12 derniers mois.
- **Total\_Ct\_Chng\_Q4\_Q1:** Variation du nombre de transactions entre le quatrième et le premier trimestre.
- **Avg\_Utilization\_Ratio:** Taux moyen d'utilisation des cartes.

Ces features fournissent des informations sur les caractéristiques démographiques et le comportement des clients, qui sont cruciales pour l'élaboration de modèles prédictifs.

## Data Manipulation and Exploration

Le data munging, ou data wrangling, est le processus de nettoyage et de transformation des données brutes dans un format utilisable. Cette étape est cruciale pour préparer l'ensemble de données à une analyse plus approfondie et à la construction de modèles. L'objectif du data munging est de traiter les valeurs manquantes, de formater les données de manière appropriée et de s'assurer que toutes les caractéristiques sont adaptées aux algorithmes d'apprentissage automatique.

```
In [22]: # Vérifier les valeurs uniques dans la colonne cible 'Attrition_Flag'
cust_df['Attrition_Flag'].unique()
```

```
Out[22]: array(['Existing Customer', 'Attrited Customer'], dtype=object)
```

# Exploration et Préparation des données

Dans cette section, nous effectuons plusieurs opérations afin de préparer les données pour la modélisation :

- **Traitement des valeurs manquantes** : Il est essentiel de vérifier la présence de valeurs manquantes dans l'ensemble de données, car elles peuvent avoir un impact négatif sur les performances du modèle.
- **Transformer les caractéristiques catégorielles** : Convertir les variables catégorielles en représentations numériques pouvant être traitées par des algorithmes d'apprentissage automatique.
- **Traitement du déséquilibre entre les classes** : Traiter tout déséquilibre entre les classes afin d'améliorer les performances des modèles prédictifs.

## Étapes de la manipulation des données

L'ensemble de données contient deux valeurs dans la colonne cible "Attrition\_Flag" :

- **Existing Customer** : Représente les clients qui n'ont pas changé de fournisseur.
- **Attrited Customer** : Représente les clients qui ont abandonné.

Nous convertissons la colonne « Attrition\_Flag » en un format binaire :

```
In [23]: # Définir le taux de désabonnement comme 1 (Oui) et 0 (Non)
cust_df["Churn"] = np.where(cust_df["Attrition_Flag"] == "Attrited Customer", 1, 0)

# Supprimer la colonne 'Attrition_Flag' d'origine
cust_df.drop(columns=["Attrition_Flag"], inplace=True)
```

Ensuite, nous vérifions s'il y a des valeurs manquantes dans l'ensemble de données :

```
In [24]: # Check for null values in the dataset
cust_df.isnull().sum()
```

```
Out[24]: Customer_Age      0
        Gender            0
        Dependent_count   0
        Education_Level   0
        Marital_Status     0
        Income_Category    0
        Card_Category      0
        Months_on_book     0
        Total_Relationship_Count  0
        Months_Inactive_12_mon  0
        Contacts_Count_12_mon  0
        Credit_Limit       0
        Total_Revolving_Bal  0
        Avg_Open_To_Buy    0
        Total_Amt_Chng_Q4_Q1  0
        Total_Trans_Amt    0
        Total_Trans_Ct     0
        Total_Ct_Chng_Q4_Q1  0
        Avg_Utilization_Ratio  0
        Churn              0
        dtype: int64
```

**Observation** : L'ensemble de données ne contient pas de valeurs manquantes, ce qui simplifie le processus de prétraitement.

```
In [25]: # Vérifier les valeurs uniques dans 'Income_Category'
cust_df['Income_Category'].unique()
```

```
Out[25]: array(['$60K - $80K', 'Less than $40K', '$80K - $120K', '$40K - $60K',
               '$120K +', 'Unknown'], dtype=object)
```

**Nous traitons les catégories "inconnues" de la caractéristique "catégorie de revenu" en les remplaçant par le mode (valeur la plus fréquente) de la caractéristique**

```
In [26]: # Remplacer les revenus « Inconnu » par le mode de la « Catégorie_de_revenus
cust_df['Income_Category'].replace('Unknown', cust_df['Income_Category'].mod
```

Cela permet de s'assurer que toutes les entrées de la caractéristique **Income Category** sont significatives, ce qui améliore la capacité du modèle à apprendre des modèles à partir des données.

Cela permet de s'assurer que toutes les entrées de la caractéristique **Income Category** sont significatives, ce qui améliore la capacité du modèle à apprendre des modèles à partir des données.

# EDA (Exploratory Data Analysis)

L'analyse exploratoire des données (AED) est une étape essentielle de tout projet de science des données. Elle permet de comprendre la structure de l'ensemble de données, les relations entre les caractéristiques et les modèles sous-jacents. Grâce à l'AED, nous pouvons également identifier les problèmes potentiels, tels que les valeurs aberrantes, les valeurs manquantes ou les distributions asymétriques, qu'il pourrait être nécessaire de résoudre avant de construire un modèle prédictif.

## % Customer Churn

**Visualisation de la variable cible (distribution du taux de désabonnement)** : Il est essentiel de comprendre la distribution de la variable cible (taux d'attrition) pour évaluer l'équilibre entre les classes et orienter les étapes de prétraitement ultérieures.

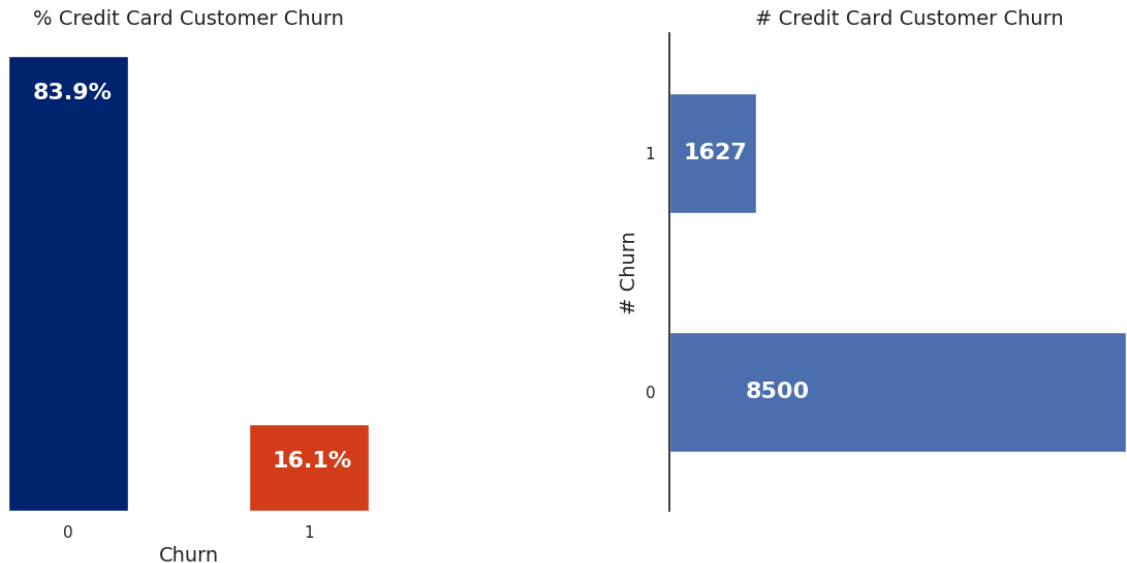
```
In [31]: from matplotlib import gridspec
# set figure and size
fig = plt.figure()
fig.set_figheight(6)
fig.set_figwidth(15)

# geometry(Grid we created for subplots),
spec = gridspec.GridSpec(ncols=2, nrows=1,
                          wspace=0.5,
                          hspace=0.5)

# ax1 will take first column
ax1 = fig.add_subplot(spec[0])
bar_plot(cust_df, ax1, column_name= "Churn", xlabel= "Churn", ylabel= "% Churn

# ax2 will take second column
ax2 = fig.add_subplot(spec[1])
bar_plot_tot(cust_df, ax2, column_name= "Churn", xlabel= "Churn", ylabel= "# Ch
print("Nous avons 16,1 % de clients qui se sont désabonnés.")
```

Nous avons 16,1 % de clients qui se sont désabonnés.



**Nous avons 16,1 % de clients qui ont abandonné. Ainsi, il est un peu difficile d'entraîner notre modèle à prédire les clients qui se désabonnent. Il s'agit d'un ensemble de données déséquilibré.**

**Observation :** L'ensemble de données sur le taux de désabonnement montre un déséquilibre entre les classes, avec davantage de clients étiquetés comme « existants » par rapport à « attrités ». Nous traiterons ce déséquilibre plus tard en utilisant la technique SMOTE (Synthetic Minority Over-sampling Technique) pendant la phase de prétraitement des données.

## Analyse des caractéristiques démographiques

Les caractéristiques démographiques telles que l'âge, le sexe, le niveau d'éducation et le revenu jouent un rôle crucial dans la compréhension du comportement des clients et la prévision du taux de désabonnement. En analysant ces caractéristiques, nous pouvons identifier des modèles et des tendances qui révèlent quels groupes sont les plus susceptibles de quitter un service.

Cette compréhension permet aux entreprises d'adapter leurs stratégies pour fidéliser les clients à haut risque, améliorer la satisfaction client et optimiser les efforts marketing. Dans cette analyse, nous examinerons l'impact de différents facteurs démographiques sur la perte de clientèle, en fournissant des informations sur les domaines clés pour des interventions ciblées.

## % de désabonnement par niveau d'éducation

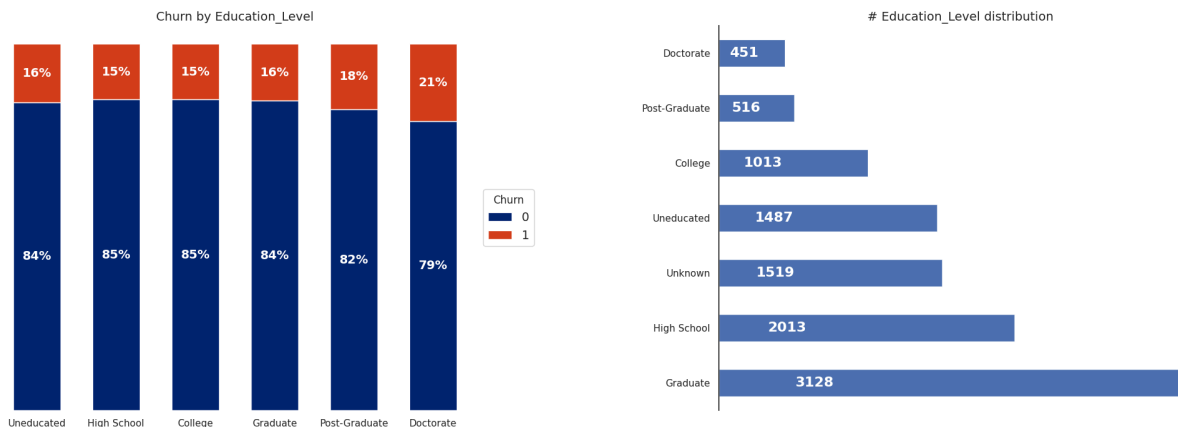
```
In [30]: from matplotlib import gridspec
# set figure and size
fig = plt.figure()
fig.set_figheight(8)
fig.set_figwidth(25)

# geometry(Grid we created for subplots),
spec = gridspec.GridSpec(ncols=2, nrows=1,
                          wspace=0.5,
                          hspace=0.5)

#reorder the categories
reorderlist = ['Uneducated', 'High School', 'College', 'Graduate', 'Post-Graduate', 'Doctorate']

# ax1 will take first column
ax1 = fig.add_subplot(spec[0])
bar_stacked_plot(cust_df, ax1, label= 'Churn', feature= 'Education_Level', title= 'Churn by Education_Level')

# ax2 will take second column
ax2 = fig.add_subplot(spec[1])
bar_plot_tot(cust_df, ax2, column_name= 'Education_Level', title= '# Education_Level distribution')
```



Les titulaires d'un diplôme de troisième cycle et de doctorat ont tendance à se désabonner un peu plus.

**Les clients ayant un niveau d'éducation plus élevé sont plus susceptibles de se désabonner, même s'ils sont moins nombreux que ceux ayant un niveau d'éducation plus faible.**

## % de désabonnement par sexe

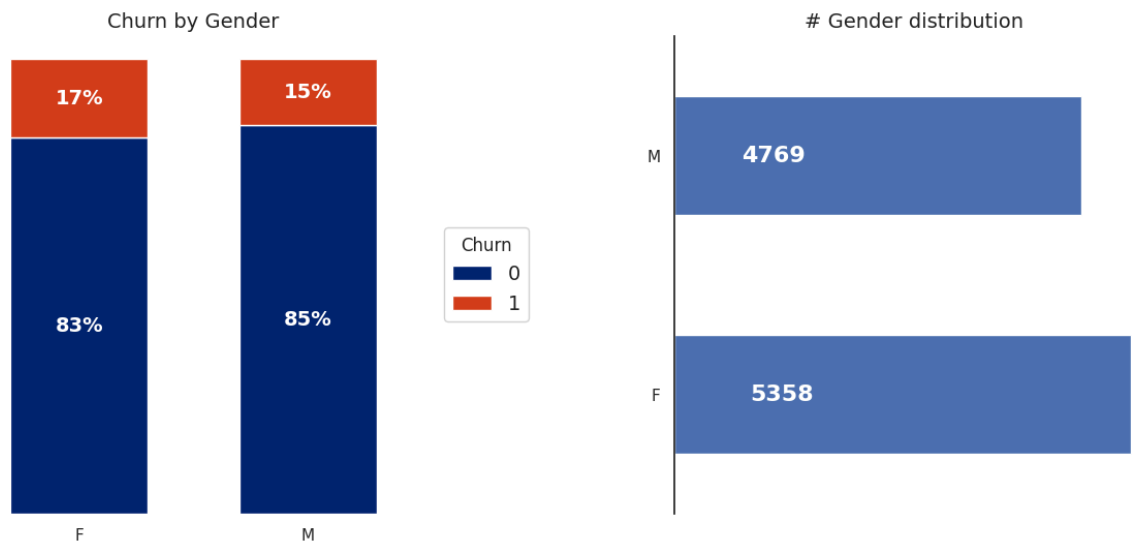
```
In [32]: from matplotlib import gridspec
# set figure and size
fig = plt.figure()
fig.set_figheight(6)
```

```
fig.set_figwidth(15)

# geometry(Grid we created for subplots),
spec = gridspec.GridSpec(ncols=2, nrows=1,
                          wspace=0.5,
                          hspace=0.5)

# ax1 will take first column
ax1 = fig.add_subplot(spec[0])
bar_stacked_plot(cust_df, ax1, label= 'Churn', feature= 'Gender', title= 'Churn by Gender')

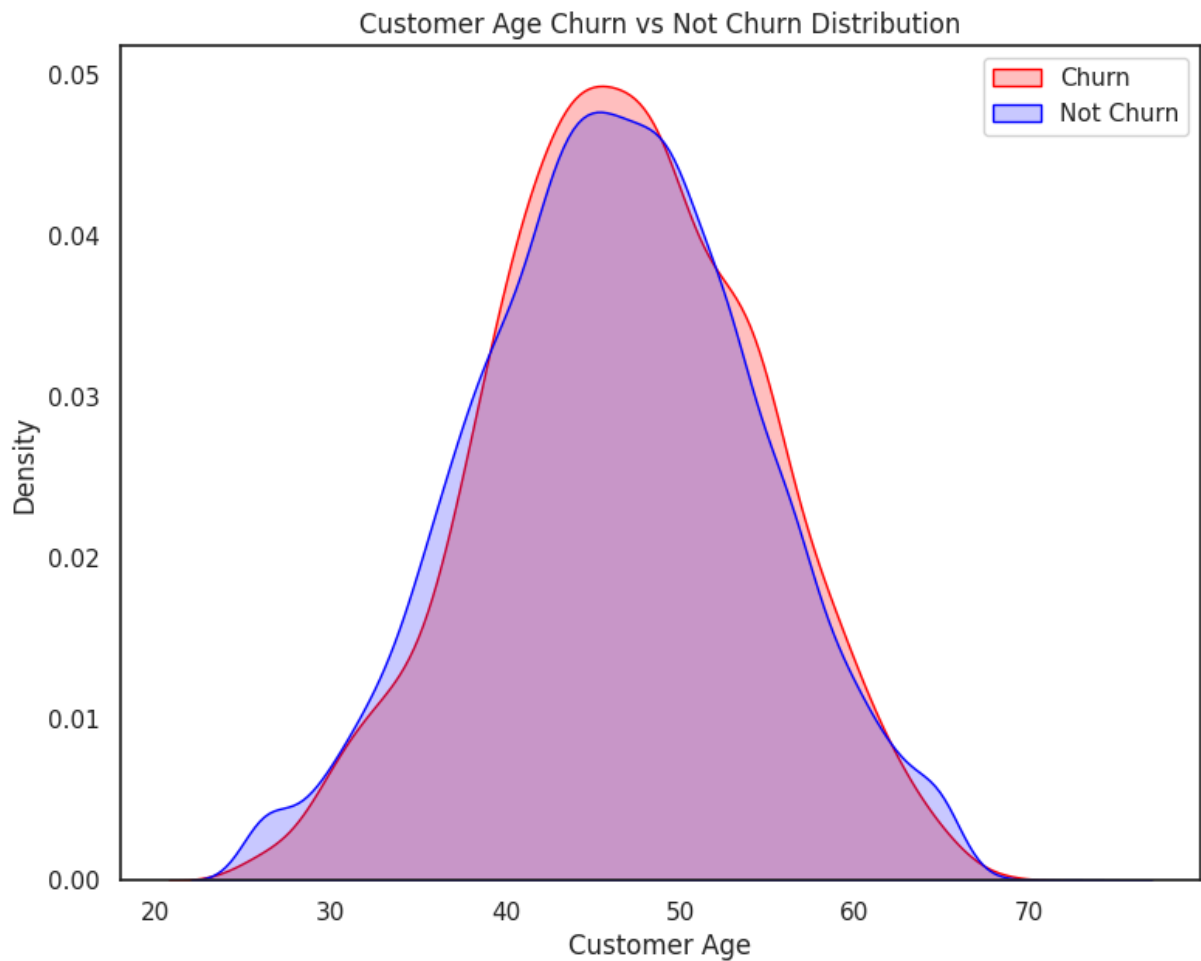
# ax2 will take second column
ax2 = fig.add_subplot(spec[1])
bar_plot_tot(cust_df, ax2, column_name= 'Gender', title= "# Gender distribution")
```



**Les hommes ont tendance à être un peu plus nombreux à fréquenter les clients. Les femmes constituent la majorité,**

## Répartition du taux de désabonnement par âge

```
In [34]: fig, ax = plt.subplots(figsize=(9,7))
dist_plot_kde(df = cust_df, ax = ax, label = 'Churn', feature = "Customer_Age")
```



n'existe pas de forte corrélation entre l'âge et le taux de désabonnement

Le graphique montre la distribution des âges des clients pour deux groupes : ceux qui ont changé de client (en rouge) et ceux qui n'ont pas changé de client (en bleu).

### Observations clés :

1. **Tranche d'âge :** La répartition par âge des deux groupes est assez similaire, allant d'environ 20 à 70 ans.
2. **Modèle de désabonnement :** les pics de densité pour les deux groupes se produisent autour de 40 à 50 ans, ce qui indique que la plupart des clients, quel que soit leur statut de désabonnement, se situent dans cette tranche d'âge.
3. **Légère différence dans la distribution :**
  - Les clients qui se désabonnent (ligne rouge) ont un pic de densité légèrement plus élevé que ceux qui ne se désabonnent pas (ligne bleue), ce qui suggère une proportion légèrement plus élevée de clients qui se désabonnent dans cette tranche d'âge.

- Cependant, dans l'ensemble, les différences entre les deux distributions sont minimales, ce qui indique que l'âge n'est peut-être pas un facteur significatif influençant le taux de désabonnement dans cet ensemble de données.

Le graphique suggère qu'il n'existe pas de forte corrélation entre l'âge et le taux de désabonnement, car les deux distributions sont étroitement alignées.

## Analyse des caractéristiques comportementales

Les caractéristiques comportementales, telles que la fréquence des transactions, les habitudes de dépenses, l'utilisation du crédit et les niveaux d'engagement, fournissent des informations essentielles sur les habitudes et les préférences des clients.

En examinant ces caractéristiques, nous pouvons mieux comprendre quels comportements sont liés à une probabilité plus élevée de désabonnement. Cette analyse permet d'identifier les signes avant-coureurs d'un désabonnement potentiel, ce qui permet d'élaborer des stratégies proactives pour fidéliser les clients. La compréhension du comportement des clients permet aux entreprises de personnaliser leurs offres, d'améliorer l'expérience client et, en fin de compte, de réduire les taux de désabonnement. Dans cette section, nous explorerons comment divers facteurs comportementaux influencent la fidélisation et la rétention des

### % de désabonnement par revenu

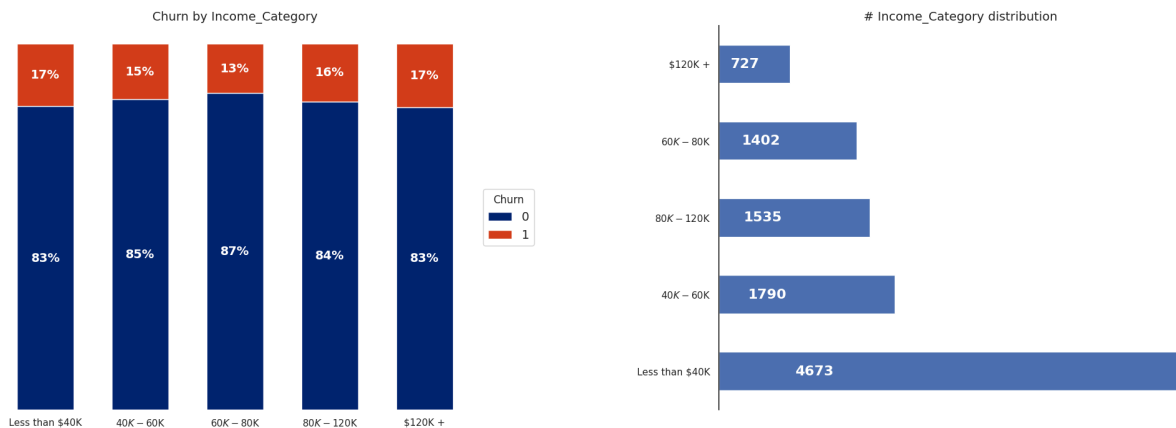
```
In [17]: from matplotlib import gridspec
# set figure and size
fig = plt.figure()
fig.set_figheight(8)
fig.set_figwidth(25)

# geometry(Grid we created for subplots),
spec = gridspec.GridSpec(ncols=2, nrows=1,
                        wspace=0.5,
                        hspace=0.5)

#reorder the categories
reorderlist = ['Less than $40K', '$40K - $60K', '$60K - $80K', '$80K - $120K']

# ax1 will take first column
ax1 = fig.add_subplot(spec[0])
bar_stacked_plot(cust_df, ax1, label= 'Churn', feature= 'Income_Category', ti
```

```
# ax2 will take second column
ax2 = fig.add_subplot(spec[1])
bar_plot_tot(cust_df, ax2, column_name= 'Income_Category', title= "# Income_C
```



**Aperçu général :** Il n'existe pas de modèle cohérent selon lequel les revenus les plus élevés ou les plus faibles seraient directement liés aux taux de désabonnement. Les catégories de revenus les plus faibles et les plus élevées présentent les taux de désabonnement les plus élevés.

## % de désabonnement par catégorie de carte

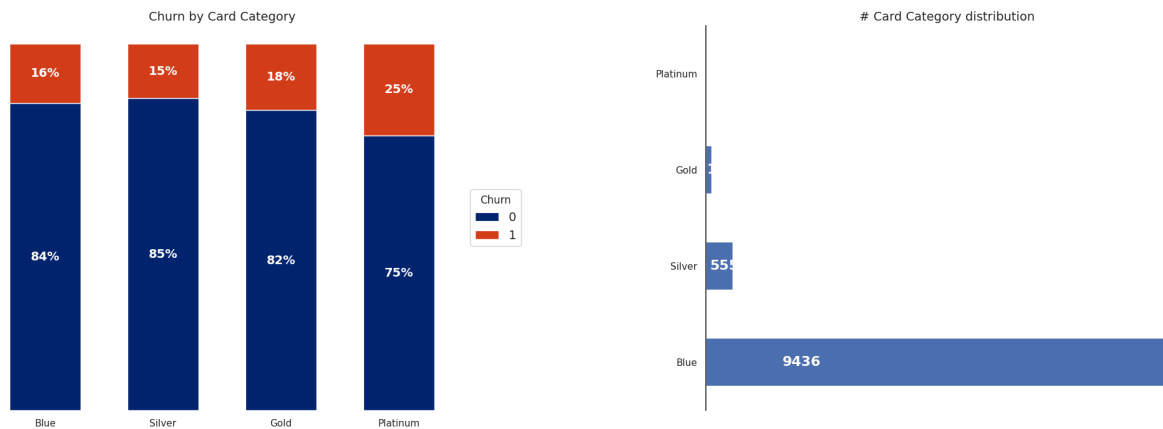
```
In [18]: # set figure and size
fig = plt.figure()
fig.set_figheight(8)
fig.set_figwidth(25)

# geometry(Grid we created for subplots),
spec = gridspec.GridSpec(ncols=2, nrows=1,
                          wspace=0.5,
                          hspace=0.5)

#reorder the categories
reorderlist = ['Blue', 'Silver', 'Gold', 'Platinum']

# ax1 will take first column
ax1 = fig.add_subplot(spec[0])
bar_stacked_plot(cust_df, ax1, label= 'Churn', feature= 'Card_Category', titl

# ax2 will take second column
ax2 = fig.add_subplot(spec[1])
bar_plot_tot(cust_df, ax2, column_name= 'Card_Category', title= "# Card Categ
```



Le graphique fournit une analyse des taux de désabonnement par catégorie de carte et la répartition des clients entre ces catégories. **Graphique de gauche : taux de désabonnement par catégorie de carte (graphique à barres empilées)**

- Catégories de cartes : Le tableau montre quatre catégories de cartes de crédit : Bleu, Argent, Or et Platine . **Taux de désabonnement :**
- Les clients possédant la carte Platinum ont le taux de désabonnement le plus élevé, soit 25 % .
- Les titulaires de cartes Gold suivent avec un taux de désabonnement de 18 %
- Les titulaires de cartes Bleues et Argent ont des taux de désabonnement relativement plus faibles, respectivement de 16 % et 15 %

**Aperçu général :** les titulaires de cartes de niveau supérieur (Gold et Platinum) sont plus susceptibles de se désabonner que ceux qui possèdent des cartes de niveau inférieur (Bleu et Argent).

**Graphique de droite : Distribution des catégories de cartes (graphique à barres horizontales)**

Il existe un déséquilibre important dans le nombre de clients entre les différentes catégories de cartes, la plupart des clients étant regroupés dans la carte de niveau le plus bas (bleue). **Des aperçus combinés**

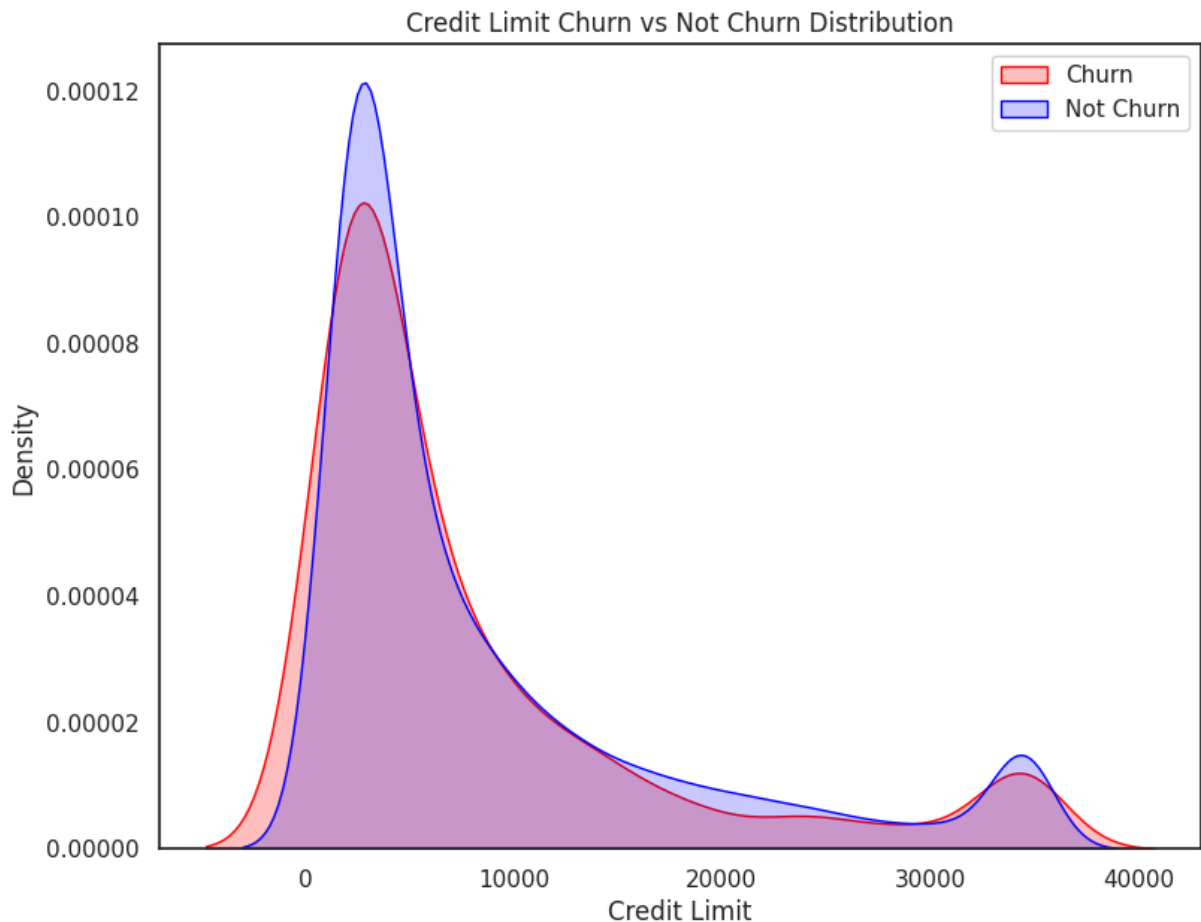
- **Concentration de désabonnement dans les niveaux supérieurs :** les taux de désabonnement plus élevés pour les titulaires de cartes Gold et Platinum suggèrent que les clients possédant des cartes plus premium peuvent être plus insatisfaits ou avoir des attentes plus élevées qui ne sont pas satisfaites.
- **Dominance des titulaires de cartes de niveau inférieur :** La grande majorité des clients titulaires de la carte bleue indiquent une concentration sur les produits d'entrée de gamme, où les taux de désabonnement sont relativement plus faibles.

- Dans l'ensemble, l'analyse met en évidence une tendance au désabonnement plus élevée parmi les titulaires de cartes premium, malgré leur nombre plus restreint, ce qui suggère la nécessité de stratégies de fidélisation ciblées pour les clients possédant des cartes de grande valeur.

In [ ]:

## Distribution du taux d'attrition des limites de crédit

```
In [20]: fig, ax = plt.subplots(figsize=(9,7))
dist_plot_kde(df = cust_df, ax = ax, label = 'Churn', feature = "Credit_Limit")
```



Le graphique illustre la distribution des limites de crédit pour deux groupes de clients : ceux qui ont changé de fournisseur (en rouge) et ceux qui n'ont pas changé de fournisseur (en bleu).

### Principales observations :

#### 1. Credit Limit Distribution:

- Les distributions des clients ayant changé de banque et des clients n'ayant pas changé de banque présentent un pic pour les limites de crédit les plus

basses, autour de 5 000 \$ ou moins. Cela suggère qu'un nombre significatif de clients, indépendamment de leur statut de résiliation, ont des limites de crédit relativement basses.

- Un second pic, plus petit, est visible autour de 30 000 à 40 000 dollars pour les deux groupes, indiquant un autre groupe de clients avec des limites de crédit plus élevées.

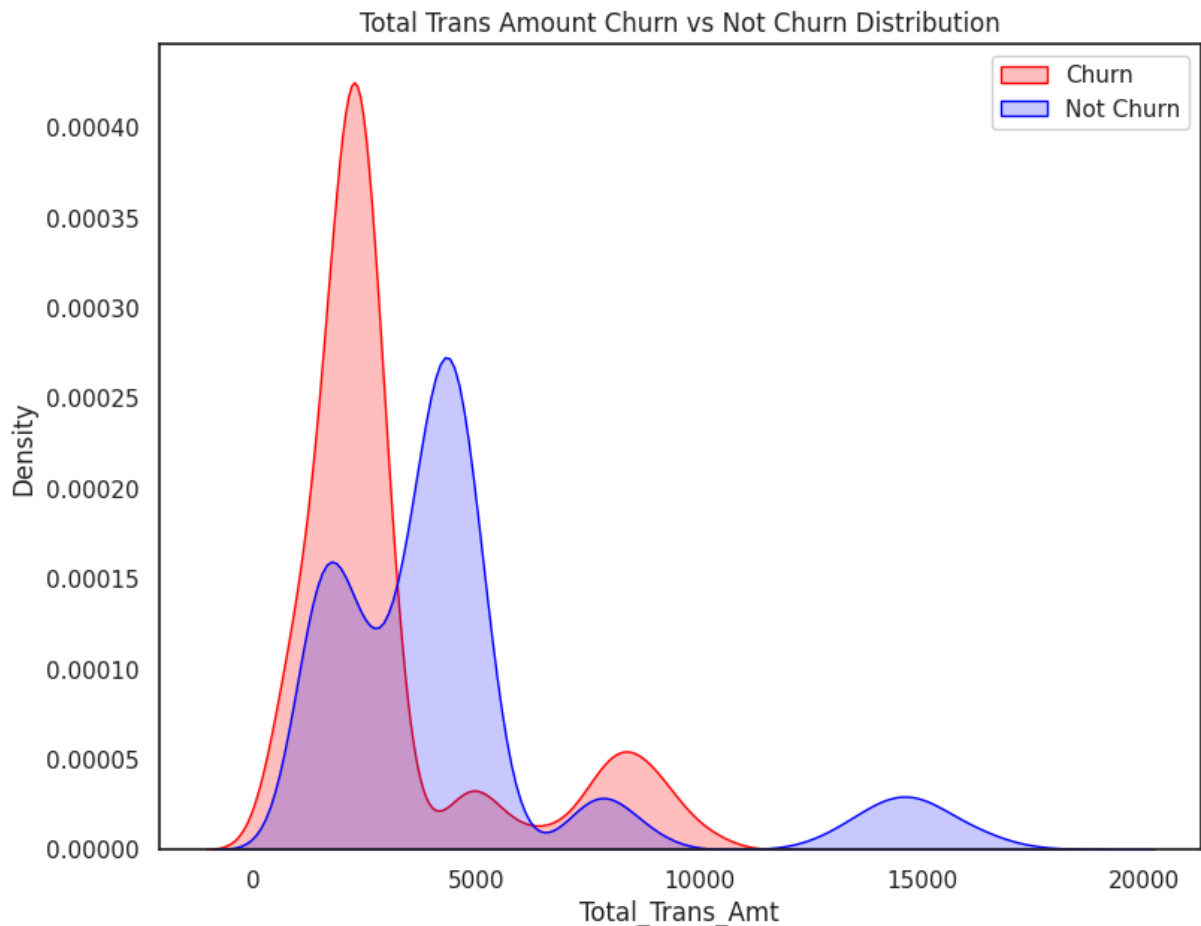
## 2. Modèle de désabonnement :

- La densité des clients non résiliés (bleu) est légèrement plus élevée que celle des clients résiliés (rouge) à l'extrémité inférieure de l'éventail des limites de crédit, en particulier autour du premier pic.
- Au fur et à mesure que la limite de crédit augmente au-delà de 10 000 \$, la différence entre les clients qui renouvellent leur contrat et ceux qui ne le font pas devient moins prononcée, avec des schémas de distribution similaires jusqu'au pic secondaire.

Cette analyse suggère que, bien que les clients ayant une limite de crédit très basse aient légèrement tendance à moins se désabonner, la limite de crédit n'est peut-être pas un facteur décisif pour prédire le désabonnement des clients au-delà d'une certaine fourchette.

## Répartition des désabonnements par montant total des transactions¶

```
In [21]: fig, ax = plt.subplots(figsize=(9,7))
dist_plot_kde(df = cust_df, ax = ax, label = 'Churn', feature = "Total_Trans_A
```



Le graphique montre la **distribution des montants totaux des transactions** pour deux groupes de clients : ceux qui ont changé de fournisseur (en rouge) et ceux qui n'ont pas changé de fournisseur (en bleu).

Principales observations :

1. Distribution du montant de la transaction : \*\*

- **Churned Customers (Red)**(Clients résiliés (rouge)) : La distribution des clients résiliés présente un pic important pour un montant total de transaction relativement faible, de l'ordre de 2 500 \$ ou moins. Cela indique que la plupart des clients qui se désabonnent ont des montants de transaction totaux inférieurs.
- **Non-Churned Customers (Blue)** Clients non fidélisés (bleu): La distribution pour les clients n'ayant pas renouvelé leur commande présente également un pic autour de la même fourchette, mais elle est plus étalée. En outre, il existe un second pic notable autour de 5 000 \$, ce qui suggère que les clients non résiliés ont tendance à effectuer des transactions d'un montant plus élevé

2. Modèle de désabonnement :

- Les clients qui ont changé de fournisseur ont généralement des montants totaux de transaction plus faibles que ceux qui n'ont pas changé de fournisseur. La densité des clients ayant changé de fournisseur est nettement plus élevée pour les montants les plus faibles et diminue fortement à mesure que le montant de la transaction augmente.
- En revanche, les clients qui n'ont pas changé de fournisseur présentent une distribution plus large pour les différents montants de transaction, allant bien au-delà de 10 000 dollars.

### 3. Aperçu général:

- Il existe une corrélation négative évidente entre le montant total des transactions et la probabilité de désabonnement : les clients dont le montant total des transactions est moins élevé sont plus susceptibles de se désabonner.
- Des montants de transaction plus élevés sont associés à un taux de désabonnement plus faible, ce qui indique que les clients les plus engagés (ceux qui dépensent plus) sont moins susceptibles de partir.

Cette analyse suggère qu'encourager des montants de transaction plus élevés pourrait être un axe stratégique pour réduire le taux d'attrition, étant donné qu'une activité de transaction plus faible est fortement associée à une probabilité plus élevée d'attrition de la clientèle.

## CORRELATION MATRIX

La matrice des corrélations La carte thermique fournit une représentation visuelle de la matrice des corrélations pour diverses caractéristiques de l'ensemble de données client. Les couleurs indiquent la force et la direction des relations entre les différentes variables.

Les couleurs indiquent la force et la direction des relations entre les différentes variables. Cette analyse des corrélations permet d'identifier les relations clés qui peuvent être exploitées pour la modélisation prédictive et une compréhension plus approfondie des comportements des clients qui influencent le désabonnement.

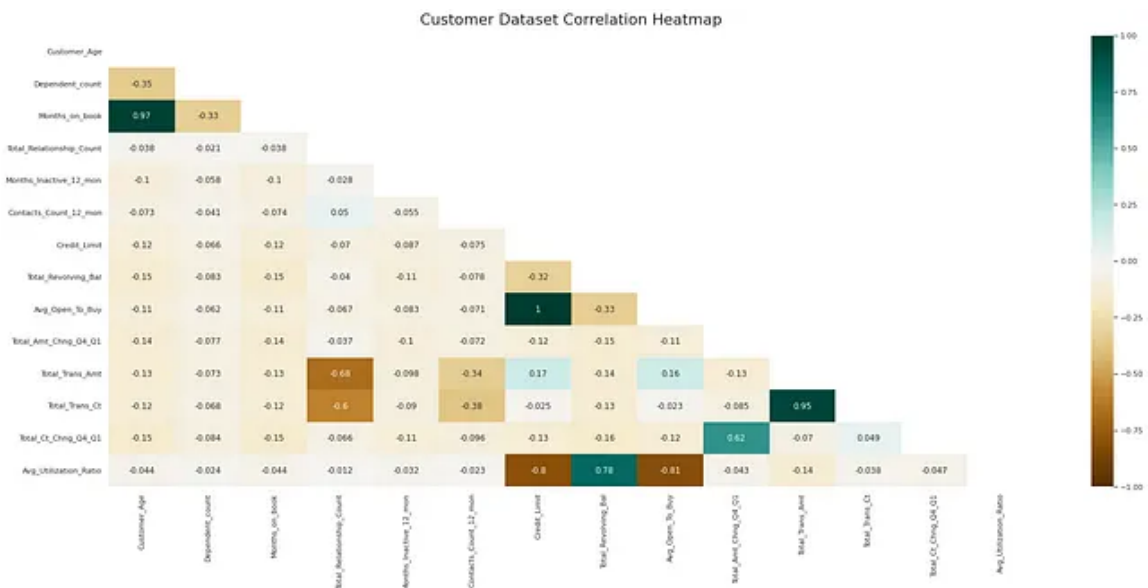
La matrice de corrélation est essentielle pour identifier toutes les corrélations possibles en une seule fois, éliminant ainsi la nécessité d'examiner chaque paire de variables individuellement. Elle fournit une vue d'ensemble de la manière dont les différentes caractéristiques sont liées les unes aux autres.

```
In [ ]: #select numerical feature and exclude the label
num_data = cust_df.select_dtypes(exclude=['object'])
df_corr_matrix = num_data.iloc[:, :-1].corr()

#if correlation is low put 0 (for correlation matrix readability)
df_corr_matrix[abs(df_corr_matrix) < 0.1] = 0

#plot the correlation matrix
plt.figure(figsize=(30, 12))

# define the mask to set the values in the upper triangle to True
mask = np.triu(np.ones_like(df_corr_matrix.corr(), dtype=np.bool_))
heatmap = sns.heatmap(df_corr_matrix.corr(), mask=mask, vmin=-1, vmax=1, annot=True,
heatmap.set_title('Customer Dataset Correlation Heatmap', fontdict={'fontsize': 16, 'fontweight': 'bold'})
```



## Considerations

### Observations clés :

#### 1. Fortes corrélations positives :

**Customer\_Age et Months\_on\_book(0,97) :** Il existe une très forte corrélation positive entre l'âge du client et le nombre de mois dans le carnet d'adresses, ce qui suggère que les clients plus âgés ont tendance à avoir une plus longue ancienneté dans l'entreprise.

**Total\_Revolving\_Bal et Avg\_Utilization\_Ratio(0,78) :** Cette forte corrélation positive indique qu'à mesure que le solde renouvelable total augmente, le taux d'utilisation moyen augmente également.

#### 2. Corrélations positives modérées :

- \*\*\*Total\_Trans\_Ct et Total\_Amt\_Chng\_Q4\_Q1(0,62) :\*\*\* Cela suggère que les clients qui ont une variation plus élevée du montant des transactions du T4 au T1 ont également tendance à avoir un nombre plus élevé de transactions.

- **Total\_Trans\_Amtet Total\_Trans\_Ct(0,6)** : indique qu'un nombre plus élevé de transactions est associé à un montant total de transactions plus élevé.

### 3. Fortes corrélations négatives :

- **Avg\_Open\_To\_Buyet Avg\_Utilization\_Ratio(-0,81)** : Cette forte corrélation négative indique que les clients disposant d'une ligne de - crédit ouverte à l'achat plus élevée ont tendance à avoir un taux d'utilisation plus faible.
- **Credit\_Limitet Total\_Revolving\_Bal(-0,8)** : Il existe une corrélation négative entre la limite de crédit et le solde renouvelable total, ce qui suggère qu'à mesure que la limite de crédit augmente, le solde renouvelable a tendance à diminuer.

### 4. Autres corrélations notables :

- **Total\_Trans\_Amtet Avg\_Utilization\_Ratio(-0,68)** : Cela indique qu'à mesure que le montant total des transactions augmente, le taux d'utilisation moyen diminue.
- **Credit\_Limitet Avg\_Utilization\_Ratio(-0,8)** : suggère que les clients avec des limites de crédit plus élevées ont tendance à avoir un taux d'utilisation plus faible.

## Aperçu général :

**-Modèles comportementaux** : Il existe des relations claires entre le comportement transactionnel des clients et leur utilisation du crédit, comme le lien entre le nombre de transactions et les montants des transactions.

**-Indicateurs de risque** : une forte corrélation entre certaines caractéristiques, comme Total\_Revolving\_Bal et Avg\_Utilization\_Ratio, peut indiquer des comportements financiers plus risqués associés à des taux de désabonnement plus élevés. **-Profils clients** : les fortes corrélations entre des fonctionnalités telles que Customer\_Age et Months\_on\_book aident à définir des profils clients qui peuvent être utilisés pour des stratégies de marketing et de fidélisation ciblées.

## Considérations importantes sur la sélection et la corrélation des fonctionnalités

Au cours de la phase d'analyse de la matrice de corrélation, il a été observé que plusieurs caractéristiques de l'ensemble de données présentaient une forte corrélation avec d'autres caractéristiques.

Les caractéristiques spécifiques identifiées étaient :

- 'Limite\_de\_crédit'

- 'Bal\_tournant\_total'
- 'Moyenne\_Ouverture\_Achat'
- 'Total\_Ct\_Chng\_T4\_T1'
- « Mois\_sur\_le\_livre »
- 'Total\_Trans\_Ct'

## Justification de l'élimination des fonctionnalités

Dans de nombreux contextes d'apprentissage automatique, des caractéristiques hautement corrélées peuvent conduire à une multicolinéarité, qui peut affecter négativement certains modèles (comme la régression linéaire) en gonflant la variance des estimations de coefficients, rendant le modèle instable et difficile à interpréter. Par conséquent, il peut sembler logique d'envisager d'éliminer ces caractéristiques hautement corrélées pour simplifier le modèle et réduire les problèmes potentiels de multicolinéarité.

## Expériences avec l'élimination de fonctionnalités

Des tentatives ont été faites pour supprimer ces caractéristiques corrélées afin d'observer leur impact sur les performances du modèle. Cependant, contrairement aux attentes, les performances des modèles (par exemple, LightGBM, Random Forest) se sont dégradées lorsque ces caractéristiques ont été éliminées. Cela indique que ces caractéristiques, malgré leur forte corrélation, fournissent des informations précieuses qui contribuent au pouvoir prédictif global des modèles.

## Pourquoi une corrélation élevée n'est pas un problème pour les modèles choisis

Les types de modèles utilisés dans ce projet, notamment Random Forest et LightGBM, sont des méthodes d'ensemble basées sur des arbres. Ces modèles sont intrinsèquement robustes à la multicolinéarité pour plusieurs raisons :

1. **Aucune hypothèse de linéarité** : Contrairement à la régression linéaire, les modèles basés sur des arbres ne font pas d'hypothèses sur la relation linéaire entre les caractéristiques. Par conséquent, ils ne sont pas affectés par des corrélations élevées entre les variables d'entrée.
2. **Mécanisme d'importance des caractéristiques** : Random Forests et LightGBM gèrent les caractéristiques corrélées en prenant en compte l'importance des caractéristiques lors du processus de création de l'arbre. Si deux caractéristiques sont fortement corrélées, le modèle peut choisir une caractéristique plutôt qu'une autre lors de la division, gérant ainsi efficacement la redondance des données sans aucun impact négatif.

3. **Aucun problème de multicollinéarité** : la multicollinéarité est principalement un problème pour les modèles qui reposent sur le calcul de coefficients (par exemple, la régression linéaire, la régression logistique). En revanche, les arbres de décision et les méthodes d'ensemble comme Random Forests et LightGBM fonctionnent en partitionnant l'espace des caractéristiques et ne produisent pas de coefficients qui peuvent être déstabilisés par la multicollinéarité. Ainsi, la présence de caractéristiques hautement corrélées n'affecte pas leurs performances.

## Conclusion

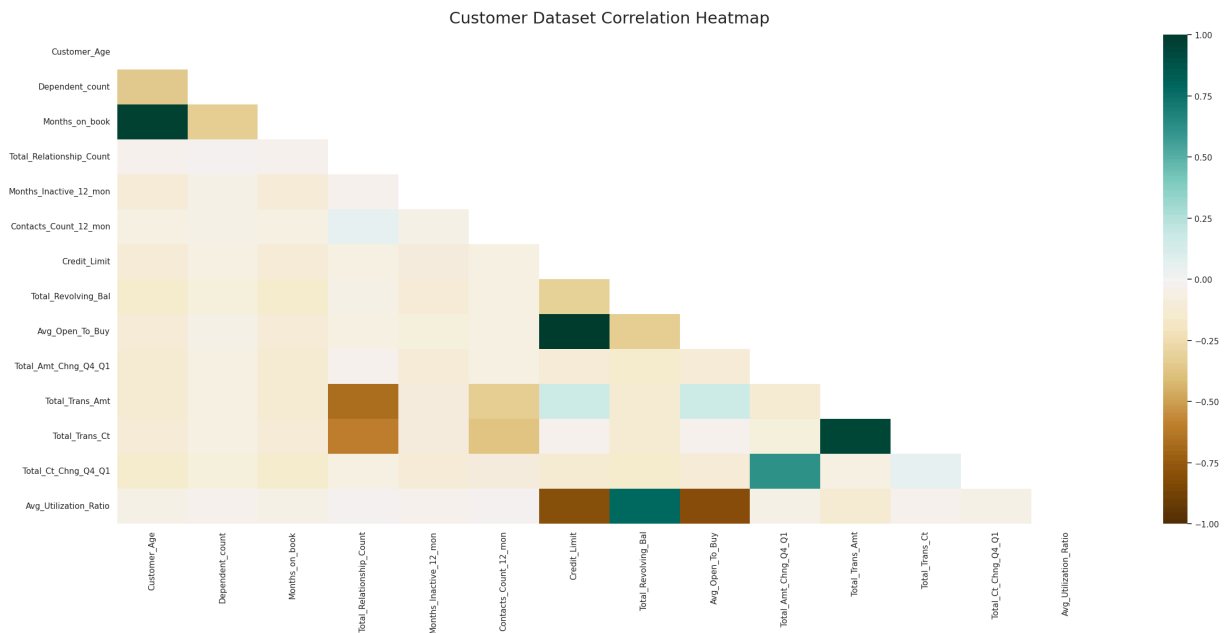
Sur la base des résultats expérimentaux et des propriétés des modèles utilisés, il a été déterminé que la suppression de caractéristiques fortement corrélées a un impact négatif sur les performances du modèle. Par conséquent, malgré la forte corrélation entre certaines caractéristiques, celles-ci sont conservées dans le modèle. Les modèles choisis sont bien équipés pour gérer les caractéristiques corrélées, en exploitant leurs informations sans être affectés par la multicollinéarité. Cela conduit à de meilleures performances prédictives et garantit que toutes les données pertinentes sont utilisées efficacement.

```
In [22]: #select numerical feature and exclude the label
num_data = cust_df.select_dtypes(exclude=['object'])
df_corr_matrix = num_data.iloc[:, :-1].corr()

#if correlation is low put 0 (for correlation matrix readability)
df_corr_matrix[abs(df_corr_matrix) < 0.1] = 0

#plot the correlation matrix
plt.figure(figsize=(30, 12))

# define the mask to set the values in the upper triangle to True
mask = np.triu(np.ones_like(df_corr_matrix.corr(), dtype=np.bool_))
heatmap = sns.heatmap(df_corr_matrix.corr(), annot=True, mask=mask, vmin=-1,
heatmap.set_title('Customer Dataset Correlation Heatmap', fontdict={'fontsize': 16})
```



## Principales observations :

### 1. Corrélations positives fortes :\*\*

- **Customer\_Age and Months\_on\_book (0.97):** Il existe une très forte corrélation positive entre l'âge du client et le nombre de mois d'ancienneté, ce qui suggère que les clients plus âgés ont tendance à rester plus longtemps dans l'entreprise.
- **Total\_Revolving\_Bal and Avg\_Utilization\_Ratio (0.78):** Cette forte corrélation positive indique qu'à mesure que le solde total des crédits renouvelables augmente, le taux d'utilisation moyen augmente également.

### 2. Moderate Positive Correlations:

- **Total\_Trans\_Ct and Total\_Amt\_Chng\_Q4\_Q1 (0.62):** Cela suggère que les clients dont le montant des transactions a davantage évolué entre le quatrième et le premier trimestre ont également tendance à effectuer un plus grand nombre de transactions.
- **Total\_Trans\_Amt and Total\_Trans\_Ct (0.6):** Indique qu'un nombre élevé de transactions est associé à un montant total de transactions plus élevé.

### 3. Corrélations négatives fortes :

- **Avg\_Open\_To\_Buy and Avg\_Utilization\_Ratio (-0.81):** Cette forte corrélation négative indique que les clients dont la ligne de crédit ouverte à l'achat est plus élevée ont tendance à avoir un ratio d'utilisation plus faible.
- **Credit\_Limit and Total\_Revolving\_Bal (-0.8):** Il existe une corrélation négative entre la limite de crédit et le solde total renouvelable, ce qui suggère que plus la limite de crédit augmente, plus le solde renouvelable tend à diminuer.

#### 4. Autres corrélations notables :

- **Total\_Trans\_Amt and Avg\_Utilization\_Ratio (-0.68):** Cela indique qu'à mesure que le montant total des transactions augmente, le taux d'utilisation moyen diminue.
- **Credit\_Limit and Avg\_Utilization\_Ratio (-0.8):** Suggère que les clients ayant des limites de crédit plus élevées ont tendance à avoir un ratio d'utilisation plus faible.

#### Aperçu général :

- **Modèles de comportement :** Il existe des relations claires entre le comportement transactionnel des clients et leur utilisation du crédit, comme le lien entre le nombre de transactions et les montants des transactions.
- **Indicateurs de risque :** Une forte corrélation entre certaines caractéristiques, comme Total\_Revolving\_Bal et Avg\_Utilization\_Ratio, peut indiquer des comportements financiers plus risqués associés à des taux de résiliation plus élevés.
- **Profils des clients :** Les fortes corrélations entre des caractéristiques telles que Customer\_Age et Months\_on\_book permettent de définir des profils de clients qui peuvent être utilisés pour des stratégies de marketing et de fidélisation ciblées.

#### Considérations importantes sur la sélection des caractéristiques et la corrélation

Au cours de la phase d'analyse de la matrice de corrélation, il a été observé que plusieurs caractéristiques de l'ensemble de données présentaient une forte corrélation avec d'autres caractéristiques. **Les caractéristiques spécifiques identifiées sont les suivantes**

- 'Credit\_Limit'
- 'Total\_Revolving\_Bal'
- 'Avg\_Open\_To\_Buy'
- 'Total\_Ct\_Chng\_Q4\_Q1'
- 'Months\_on\_book'
- 'Total\_Trans\_Ct'

#### Justification de l'élimination des caractéristiques

Dans de nombreux contextes d'apprentissage automatique, les caractéristiques fortement corrélées peuvent conduire à la multicollinéarité, qui peut affecter négativement certains modèles (tels que la régression linéaire) en gonflant la variance des estimations de coefficient, ce qui rend le modèle instable et difficile à interpréter. Par conséquent, il peut sembler logique d'envisager l'élimination de

ces caractéristiques fortement corrélées afin de simplifier le modèle et de réduire les problèmes potentiels de multicollinéarité.

## Expériences d'élimination des caractéristiques

Des tentatives ont été faites pour supprimer ces caractéristiques corrélées afin d'observer leur impact sur les performances du modèle. Cependant, contrairement aux attentes, les performances des modèles (par exemple, LightGBM, Random Forest) se sont dégradées lorsque ces caractéristiques ont été éliminées. Cela indique que ces caractéristiques, malgré leur forte corrélation, fournissent des informations précieuses qui contribuent au pouvoir prédictif global des modèles.

## Pourquoi une corrélation élevée n'est pas un problème pour les modèles choisis

Les types de modèles utilisés dans ce projet, en particulier Random Forest et LightGBM, sont des méthodes d'ensemble basées sur les arbres. Ces modèles sont intrinsèquement robustes à la multicollinéarité pour plusieurs raisons :

1. **No Assumption of Linearity:** Contrairement à la régression linéaire, les modèles à base d'arbres ne font pas d'hypothèses sur la relation linéaire entre les caractéristiques. Par conséquent, ils ne sont pas affectés par des corrélations élevées entre les variables d'entrée.
2. **Feature Importance Mechanism:** Les forêts aléatoires et LightGBM gèrent les caractéristiques corrélées en tenant compte de l'importance de la caractéristique au cours du processus de construction de l'arbre. Si deux caractéristiques sont fortement corrélées, le modèle peut choisir une caractéristique plutôt qu'une autre lors de la division, ce qui permet de gérer efficacement la redondance des données sans aucun impact négatif.
3. **No Multicollinearity Issues:** La multicollinéarité est principalement un problème pour les modèles qui reposent sur le calcul de coefficients (par exemple, la régression linéaire, la régression logistique). En revanche, les arbres de décision et les méthodes d'ensemble telles que Random Forests et LightGBM fonctionnent en partitionnant l'espace des caractéristiques et ne produisent pas de coefficients susceptibles d'être déstabilisés par la multicollinéarité. Ainsi, la présence de caractéristiques fortement corrélées n'affecte pas leurs performances.

## Conclusion

Sur la base des résultats expérimentaux et des propriétés des modèles utilisés, il a été déterminé que la suppression des caractéristiques fortement corrélées a un impact négatif sur les performances du modèle. Par conséquent, malgré la forte corrélation entre certaines caractéristiques, celles-ci sont conservées dans le

modèle. Les modèles choisis sont bien équipés pour traiter les caractéristiques corrélées, en tirant parti de leurs informations sans être affectés par la multicolinéarité. Cela permet d'obtenir de meilleures performances prédictives et de garantir que toutes les données pertinentes sont utilisées de manière efficace.

## Prétraitement et sélection du modèle

### Class imbalance

Le prétraitement des données est une étape cruciale de tout projet d'apprentissage automatique. Il consiste à transformer les données brutes en un format adapté à la modélisation. Dans cette section, nous traitons le déséquilibre des classes à l'aide de la technique de suréchantillonnage des minorités synthétiques (SMOTE).

Cette section se concentre sur la construction et l'évaluation de modèles d'apprentissage automatique pour prédire le désabonnement des clients. Il s'agit de sélectionner le meilleur modèle en fonction de ses performances sur l'ensemble des données et d'appliquer des techniques pour améliorer la précision du modèle, en particulier lorsqu'il s'agit de données déséquilibrées. Les sous-sections comprennent l'évaluation de différents modèles, la construction d'un pipeline avec SMOTE pour l'équilibrage des données, et enfin la sélection du modèle le plus performant.

### Suréchantillonnage des données à l'aide de SMOTE

Le déséquilibre des classes est un problème courant dans les tâches de classification, en particulier dans la prédiction du désabonnement des clients. Dans notre ensemble de données, le nombre de clients qui se sont désabonnés ("clients désabonnés") est beaucoup plus faible que celui des clients qui ne se sont pas désabonnés ("clients existants").

*Ce déséquilibre peut entraîner un biais des modèles d'apprentissage automatique en faveur de la classe majoritaire, ce qui se traduit par des performances médiocres en matière de prédiction de la classe minoritaire (clients résiliés).*

Pour résoudre ce problème, nous utilisons la technique SMOTE (Synthetic Minority Over-sampling Technique). SMOTE génère des échantillons synthétiques

pour la classe minoritaire en interpolant les échantillons existants. Cette technique permet d'équilibrer la distribution des classes et d'améliorer la capacité du modèle à apprendre des deux classes.

### Exemple d'application de SMOTE

1. **Division des données** : Nous commençons par diviser l'ensemble de données en ensembles de formation et de test.
2. **Application de SMOTE** : SMOTE n'est appliqué qu'à l'ensemble de formation afin d'éviter les fuites de données.

```
In [ ]: from imblearn.over_sampling import SMOTE
        from sklearn.model_selection import train_test_split

        # Séparer les fonctionnalités et la variable cible
        X = cust_df.drop( 'Churn' , axis= 1 )
        y = cust_df[ 'Churn' ]

        # Diviser les données en ensembles d'entraînement et de test
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size= 0.3 , r

        # Appliquer SMOTE à l'ensemble d'entraînement
        smote = SMOTE(random_state= 42 )
        X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

        # Afficher la nouvelle distribution de classe
        print ( "Distribution de classe d'origine : " )
        print (y_train.value_counts())
        print ( "\nClasse rééchantillonnée distribution : " )
        imprimer (y_train_resampled.value_counts())
```

Observation : Après l'application de SMOTE, la distribution des classes dans l'ensemble d'apprentissage devient équilibrée, ce qui améliore la capacité du modèle à apprendre les modèles associés aux clients abandonnés et non abandonnés.

SMOTE équilibre l'ensemble de données en générant des échantillons synthétiques pour la classe minoritaire, améliorant ainsi les performances du modèle sur la classe minoritaire (clients perdus).

## Preproession and Model selection

Avant d'introduire les données dans les modèles d'apprentissage automatique, il est essentiel de les prétraiter pour s'assurer qu'elles sont propres, bien formatées et prêtes à être analysées. Le prétraitement des données comprend plusieurs étapes clés, telles que la séparation des fonctionnalités et des étiquettes, l'encodage des variables catégorielles, le traitement des valeurs manquantes, et la division des ensembles de données pour garantir une bonne évaluation du modèle.

## 3.1 Division des Fonctionnalités et des Étiquettes

**Objectif** : Séparer l'ensemble de données en entités (variables indépendantes) et étiquettes (variables dépendantes).

- **Fonctionnalités (X)** : Les colonnes de l'ensemble de données utilisées pour prédire la variable cible. Elles incluent des données démographiques, transactionnelles et comportementales, telles que `Customer_Age`, `Gender`, `Credit_Limit`, `Total_Trans_Amt`, etc.
- **Étiquette (y)** : La colonne qui représente le résultat que nous souhaitons prédire. Dans ce cas, l'étiquette est `Churn`, indiquant si un client a quitté le service ( `1` ) ou non ( `0` ).

**Code** :

```
X = cust_df.drop(columns=['Churn']) # Fonctionnalités
y = cust_df['Churn']               # Étiquette
```

- Ici, `X` contient toutes les colonnes sauf la variable cible `Churn`, tandis que `y` contient uniquement la variable cible.

## 3.2 Encodage des Étiquettes

**Objectif** : Convertir les étiquettes catégorielles en un format adapté aux modèles d'apprentissage automatique, qui nécessitent généralement une saisie numérique.

Dans notre cas, `Churn` est déjà une variable binaire ( `0` ou `1` ), donc l'encodage n'est pas nécessaire. Toutefois, pour d'autres étiquettes catégorielles, on utiliserait un encodage numérique, par exemple en utilisant `LabelEncoder` ou `OneHotEncoder` si nécessaire.

## 3.3 Identification des Caractéristiques Catégorielles et Numériques

**Objectif** : Distinguer les caractéristiques catégorielles des caractéristiques numériques. Cette étape est cruciale, car des techniques de prétraitement

différentes s'appliquent à chaque type de données.

- **Caractéristiques catégorielles** : Représentent des catégories, telles que `Gender`, `Marital_Status`, `Card_Category`.
- **Caractéristiques numériques** : Représentent des quantités continues ou discrètes, telles que `Customer_Age`, `Credit_Limit`, `Total_Trans_Amt`.

**Code :**

```
cat_ix = X.select_dtypes(include=['object', 'bool']).columns #  
Caractéristiques catégorielles  
num_ix = X.select_dtypes(include=['int64', 'float64']).columns #  
Caractéristiques numériques
```

### 3.4 Traitement des Valeurs Manquantes et Encodage des Catégories

Une fois les caractéristiques catégorielles et numériques identifiées, nous devons traiter les valeurs manquantes et encoder les variables catégorielles pour qu'elles soient compatibles avec les algorithmes d'apprentissage automatique.

- **SimpleImputer** : Remplace les valeurs manquantes. Par exemple, pour les données numériques, on remplace les valeurs manquantes par la moyenne ou la médiane. Pour les données catégorielles, on utilise la valeur la plus fréquente.
- **OneHotEncoder** : Convertit les variables catégorielles en variables binaires afin de permettre aux modèles de les utiliser correctement.

**Code :**

```
# Définir un pipeline pour le traitement des données catégorielles  
categorical_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='constant',  
    fill_value='missing')),  
    ('encoder', OneHotEncoder(handle_unknown='ignore'))  
])
```

### 3.5 Division de l'Ensemble de Données en Ensembles d'Entraînement et de Test

Pour évaluer la performance d'un modèle et s'assurer qu'il peut généraliser à de nouvelles données, il est essentiel de diviser les données en un ensemble d'entraînement et un ensemble de test. Cela permet d'entraîner le modèle sur une partie des données (ensemble d'entraînement) et de l'évaluer sur des données non vues (ensemble de test).

## Code :

```
from sklearn.model_selection import train_test_split
```

```
# Diviser les données en ensembles d'entraînement et de test  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2, random_state=42)
```

- **X\_train, y\_train** : Ensemble d'entraînement utilisé pour former le modèle.
- **X\_test, y\_test** : Ensemble de test utilisé pour évaluer les performances du modèle.

## Conclusion

Ces étapes de prétraitement garantissent que les données sont propres, bien formatées et prêtes pour l'entraînement des modèles d'apprentissage automatique. Le traitement des données catégorielles et numériques, ainsi que la gestion des valeurs manquantes, sont essentiels pour améliorer la précision et la robustesse des modèles.

# Data Preprocessing

```
In [48]: ## Diviser les fonctionnalités et les étiqueter  
y = cust_df['Churn'].values  
X = cust_df.drop(columns = ['Churn'])  
  
#catégorique et numérique  
cat_ix = X.select_dtypes(include=[ 'object' , 'bool' ]).columns  
num_ix = X.select_dtypes(include=[ 'int64' , 'float64' ]).columns  
  
# define models  
models, names = get_models()  
results = list()  
  
#define pipeline for categorical transformer  
categorical_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),  
    ('encoder', OneHotEncoder(handle_unknown='ignore'))])  
  
#split train and test set  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
```

## Encode and Split

Voici un chapitre organisé qui décrit la procédure que vous avez partagée, en expliquant chaque étape et en l'intégrant dans le flux de prétraitement des données et de modélisation.

---

## Chapitre 3 : Prétraitement des Données et Modélisation

Dans cette section, nous allons détailler le processus de préparation des données et de modélisation pour prédire la variable `Churn`. Nous séparerons les fonctionnalités des étiquettes, identifierons les types de données, mettrons en place un pipeline de transformation pour les données catégorielles, puis nous entraînerons et testerons différents modèles de machine learning.

### 3.1 Division des Fonctionnalités et des Étiquettes

**Objectif** : Séparer les entités indépendantes (fonctionnalités) de la variable cible (étiquette).

- **Fonctionnalités (X)** : Les colonnes de l'ensemble de données utilisées pour prédire la variable cible. Elles incluent des données démographiques, transactionnelles et comportementales.
- **Étiquette (y)** : La colonne `Churn`, qui représente l'état de désabonnement du client (1 pour désabonné, 0 pour non désabonné).

**Code :**

```
y = cust_df['Churn'].values # Étiquette
X = cust_df.drop(columns=['Churn']) # Fonctionnalités
```

---

### 3.2 Identification des Caractéristiques Catégorielles et Numériques

**Objectif** : Identifier les caractéristiques catégorielles et numériques afin d'appliquer des techniques de prétraitement adaptées à chaque type de données.

- **Caractéristiques catégorielles** : Variables représentant des catégories (ex : `Gender`, `Card_Category`).
- **Caractéristiques numériques** : Variables représentant des valeurs continues ou discrètes (ex : `Customer_Age`, `Credit_Limit`).

**Code :**

```
# Identification des colonnes catégorielles et numériques
cat_ix = X.select_dtypes(include=['object', 'bool']).columns # Catégorielles
num_ix = X.select_dtypes(include=['int64', 'float64']).columns # Numériques
```

---

### 3.3 Définir les Modèles de Machine Learning

**Objectif** : Configurer plusieurs modèles de machine learning pour les tester et comparer leurs performances.

Nous définissons plusieurs modèles à l'aide d'une fonction `get_models()`, qui retourne une liste de modèles et leurs noms correspondants.

**Code** :

```
# Définition des modèles et des noms correspondants
models, names = get_models()
results = list() # Liste pour stocker les résultats des performances des modèles
```

---

### 3.4 Prétraitement des Caractéristiques Catégorielles avec un Pipeline

**Objectif** : Mettre en place un pipeline pour traiter les caractéristiques catégorielles, en gérant les valeurs manquantes avec `SimpleImputer` et en appliquant l'encodage avec `OneHotEncoder`.

- **SimpleImputer** : Remplace les valeurs manquantes dans les colonnes catégorielles par une valeur par défaut ( `'missing'` ).
- **OneHotEncoder** : Convertit les variables catégorielles en variables binaires pour permettre leur utilisation par les modèles d'apprentissage automatique.

**Code** :

```
# Pipeline pour le prétraitement des colonnes catégorielles
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant',
    fill_value='missing')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])
```

---

### 3.5 Division des Données en Ensembles d'Entraînement et de Test

**Objectif** : Diviser les données en deux ensembles : un pour entraîner le modèle ( `X_train`, `y_train` ), et un pour évaluer ses performances ( `X_test`, `y_test` ).

Nous utilisons la fonction `train_test_split` avec un ratio de 80% pour l'entraînement et 20% pour les tests, tout en stratifiant les données en fonction de la variable cible `Churn` pour assurer une répartition équilibrée des classes.

## Code :

```
# Division des données en ensembles d'entraînement et de test  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2, random_state=101, stratify=y)
```

- **Stratification** : Elle garantit que la répartition des classes ( `Churn` = 0 ou 1) est similaire dans les ensembles d'entraînement et de test.
  - **Random State** : Ce paramètre assure que la division des données est reproductible.
- 

## Conclusion

En suivant ces étapes, nous avons correctement séparé les fonctionnalités et l'étiquette, identifié les types de données, mis en place un pipeline de traitement pour les colonnes catégorielles, et divisé l'ensemble de données en ensembles d'entraînement et de test. Cela constitue la base essentielle pour entraîner et évaluer des modèles d'apprentissage automatique, tout en garantissant la qualité et la propreté des données.

## Explication:

### 1. Objectif de la division des données :

la `train_test_split` fonction divise l'ensemble de données en deux parties : un ensemble d'apprentissage et un ensemble de test . L'ensemble d'apprentissage ( `X_train` et `y_train`) est utilisé pour entraîner le modèle d'apprentissage automatique, lui permettant d'apprendre des modèles à partir des données. L'ensemble de test ( `X_test` et `y_test`), qui se compose de données que le modèle n'a jamais vues auparavant, est utilisé pour évaluer les performances du modèle. Cela permet d'évaluer dans quelle mesure le modèle peut se généraliser à de nouvelles données, ce qui est essentiel pour les applications du monde réel.

### 2. Proportion de la répartition des données :

le paramètre `test_size=0.2` indique que 20 % des données d'origine sont réservées aux tests, tandis que les 80 % restants sont utilisés pour l'entraînement du modèle. Il s'agit d'un ratio courant qui équilibre le besoin de données suffisantes pour entraîner le modèle tout en conservant suffisamment de données pour valider ses performances.

### 3. Assurer la reproductibilité :

le `random_state=101` paramètre garantit que la répartition entre les ensembles d'entraînement et de test est reproductible. En définissant une valeur aléatoire

fixe, la même répartition sera générée à chaque exécution du code. Cela est utile pour déboguer et comparer les résultats entre différentes exécutions ou entre différents modèles.

#### 4. Gestion des données déséquilibrées avec l'échantillonnage stratifié :

le `stratify=y` paramètre est particulièrement important dans les cas où la variable cible ( `y` ) est déséquilibrée. Par exemple, dans un problème de prédiction de perte de clientèle, il peut y avoir beaucoup moins de clients perdus que de clients non perdus. L'échantillonnage stratifié garantit que les ensembles d'entraînement et de test conservent la même proportion de chaque classe (perdus et non perdus) que dans l'ensemble de données d'origine. Cela est essentiel pour éviter que le modèle ne soit biaisé en faveur de la classe majoritaire, améliorant ainsi la fiabilité de l'évaluation des performances du modèle.

## Buid the Pipeline with SMOTE and Evaluate the Models

Encode and Split

### Chapitre 3.2 : Construire le Pipeline avec SMOTE et Évaluer les Modèles

**Objectif** : L'objectif de cette sous-section est de comparer les performances de différents modèles d'apprentissage automatique. Les modèles sont évalués à l'aide d'une validation croisée pour garantir leur robustesse et leur capacité à généraliser sur de nouvelles données.

#### Validation Croisée

La validation croisée est une méthode qui consiste à diviser les données en plusieurs sous-ensembles (ou *folds*), à entraîner le modèle sur une partie de ces sous-ensembles et à le valider sur les sous-ensembles restants. Cela permet d'évaluer la stabilité et la variance du modèle, tout en limitant les risques de surapprentissage.

#### Modèles Utilisés

Les modèles que nous comparons incluent :

- Arbres de décision
- Machines à vecteurs de support (SVM)
- Forêts aléatoires (Random Forest)
- Gradient boosting
- Bagging
- LightGBM

Chaque modèle est évalué en termes de plusieurs mesures de performance : précision, rappel, score F1, etc.

---

### 3.2.1 Suréchantillonnage avec SMOTE

Le problème des données déséquilibrées, où une classe (par exemple, les clients qui se désabonnent) est beaucoup moins représentée que l'autre, peut poser des défis pour l'entraînement des modèles. **SMOTE** (*Synthetic Minority Over-sampling Technique*) est utilisé pour résoudre ce problème en générant des échantillons synthétiques de la classe minoritaire, créant ainsi un ensemble de données plus équilibré. Cela aide à améliorer la capacité du modèle à reconnaître correctement toutes les classes.

**Code :**

```
# Initialiser SMOTE pour le suréchantillonnage de la classe
minoritaire
smt = SMOTE(random_state=42)
```

### 3.2.2 Itération sur Différents Modèles

Une boucle `for` est utilisée pour itérer sur différents modèles d'apprentissage automatique. Cela permet de comparer la performance de chaque modèle sur l'ensemble de données donné.

### 3.2.3 Prétraitement des Caractéristiques

Les étapes de prétraitement sont définies pour chaque modèle. Cela comprend :

- **Encodage des caractéristiques catégorielles** : Utilisation d'un `OneHotEncoder` pour transformer les variables catégorielles en variables binaires.
- **Mise à l'échelle des caractéristiques numériques** : Utilisation de `MinMaxScaler` pour normaliser les valeurs numériques entre 0 et 1.

**Code :**

```
# Définir les étapes de prétraitement
steps = [('c', categorical_transformer, cat_ix),
```

```
('n', MinMaxScaler(), num_ix)]
```

```
# Créer un ColumnTransformer pour appliquer les transformations  
catégorielles et numériques  
ct = ColumnTransformer(steps)
```

### 3.2.4 Création du Pipeline

Chaque modèle est encapsulé dans un **pipeline** qui inclut toutes les étapes nécessaires :

1. Prétraitement des données (transformations catégorielles et numériques).
2. Suréchantillonnage de la classe minoritaire avec SMOTE.
3. Modèle d'apprentissage automatique.

Cela permet d'automatiser l'ensemble du processus, de la transformation des données jusqu'à l'évaluation des performances.

**Code :**

```
# Itérer sur les modèles  
for i in range(len(models)):  
    # Créer un pipeline avec les étapes de transformation et le  
modèle  
    pipeline = imbpipeline(steps=[('t', ct), ('smt', smt), ('m',  
models[i])])  
  
    # Évaluer le modèle avec une validation croisée  
    scores = evaluate_model(X_train, y_train, pipeline)  
    results.append(scores)  
  
    # Afficher les performances du modèle  
    print('>%s %.3f (%.3f)' % (names[i], mean(scores),  
std(scores)))
```

### 3.2.5 Évaluation des Modèles

La fonction `evaluate_model()` est appelée pour chaque modèle dans le pipeline. Cette fonction effectue une validation croisée pour évaluer les performances du modèle et retourne les scores calculés (précision, rappel, score F1, etc.). Ces scores sont ensuite stockés pour être analysés et comparés plus tard.

**Code :**

```
# Fonction pour évaluer les performances du modèle avec validation  
croisée  
def evaluate_model(X_train, y_train, pipeline):  
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
    return cross_val_score(pipeline, X_train, y_train, cv=cv,
scoring='accuracy')
```

### 3.2.6 Tracé des Résultats

Une fois que tous les modèles ont été évalués, les résultats sont visualisés sous la forme de *boxplots*. Cela permet de comparer la distribution des scores de performance pour chaque modèle, en mettant en évidence le modèle le plus performant et celui avec la plus faible variabilité.

**Code :**

```
# Tracer les résultats des modèles
plt.figure(figsize=(15, 10))
plt.boxplot(results, labels=names, showmeans=True)
plt.title('Comparison of Model Performance')
plt.ylabel('Accuracy')
plt.show()
```

## Conclusion

Dans cette section, nous avons construit un pipeline pour le prétraitement des données, intégré SMOTE pour gérer le déséquilibre de classe, et comparé différents modèles d'apprentissage automatique à l'aide de la validation croisée. La visualisation des résultats sous forme de *boxplots* nous permet d'identifier les modèles les plus performants et les plus stables pour la prédiction du *Churn* des clients.

En intégrant SMOTE au pipeline, nous avons pu améliorer les performances des modèles sur la classe minoritaire, augmentant ainsi leur capacité à généraliser sur de nouvelles données et à identifier les clients qui risquent de se désabonner.

```
In [ ]: from imblearn.over_sampling import SMOTE
        from sklearn.model_selection import train_test_split

        # Separate the features and target variable
        X = cust_df.drop('Churn', axis=1)
        y = cust_df['Churn']

        # Split the data into training and testing sets
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran

        # Apply SMOTE to the training set
        smote = SMOTE(random_state=42)
        X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

        # Display the new class distribution
        print("Original class distribution:")
        print(y_train.value_counts())
```

```
print("\nResampled class distribution:")
print(y_train_resampled.value_counts())
```

```
In [49]: # evaluate each models

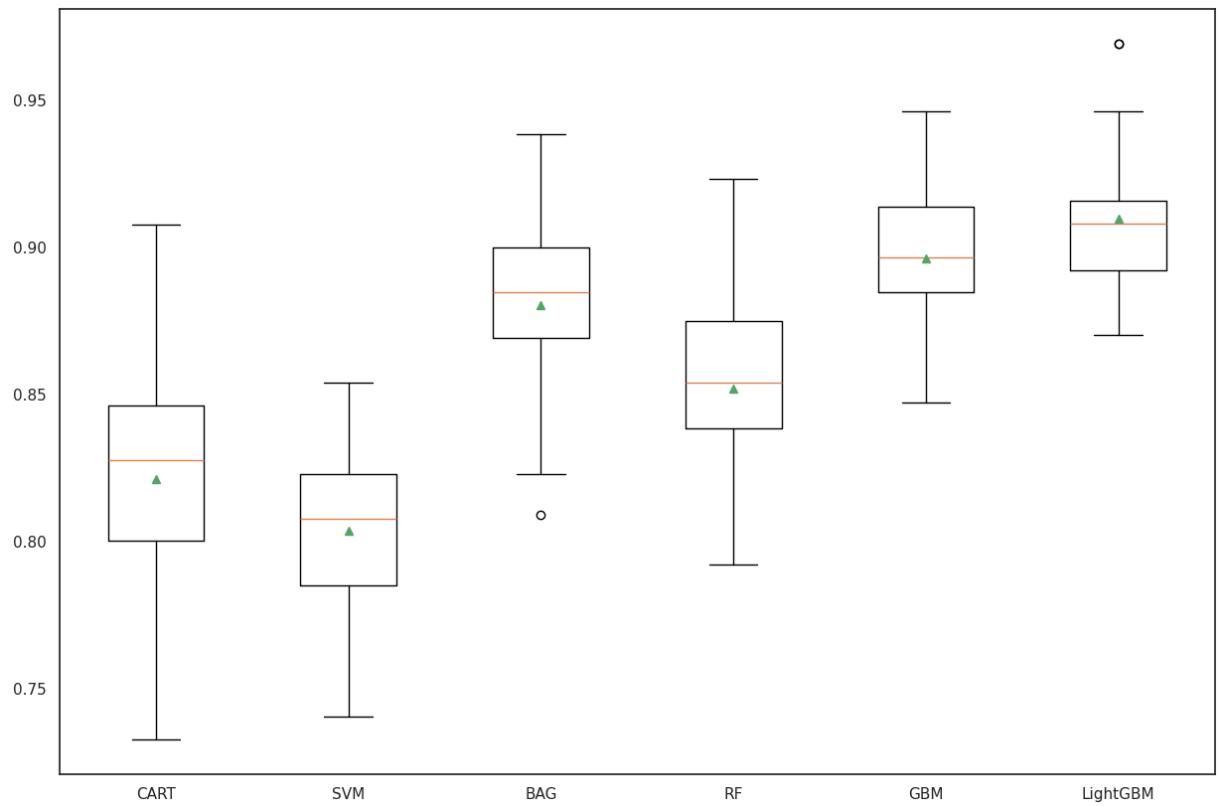
#smote for oversampling
smt = SMOTE(random_state=42)

for i in range(len(models)):
    # define steps
    steps = [('c', categorical_transformer, cat_ix), ('n', MinMaxScaler(), num_ix)]
    # one hot encode categorical, normalize numerical
    ct = ColumnTransformer(steps)
    # wrap the model i a pipeline
    pipeline = imbpipeline(steps=[('t', ct), ('smt', smt), ('m', models[i])])
    # evaluate the model and store results
    scores = evaluate_model(X_train, y_train, pipeline)
    results.append(scores)
    # summarize performance
    print('>%s %.3f (%.3f)' % (names[i], np.mean(scores), np.std(scores)))

# plot the results
plt.figure(figsize=(15, 10))
plt.boxplot(results, labels=names, showmeans=True)
plt.show()
```

```
>CART 0.821 (0.041)
>SVM 0.804 (0.027)
>BAG 0.880 (0.029)
>RF 0.852 (0.028)
>GBM 0.896 (0.023)
>LightGBM 0.910 (0.024)
```

```
/tmp/ipykernel_20506/489014154.py:22: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
plt.boxplot(results, labels=names, showmeans=True)
```



```
In [50]: print('>%s %.3f (%.3f)' % (names[i], np.mean(scores), np.std(scores)))
```

```

>LightGBM 0.910 (0.024)
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.019274 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4217
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.063792 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4173
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6120, number of negative: 6120
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.062071 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4184
[LightGBM] [Info] Number of data points in the train set: 12240, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.010383 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4146
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.004676 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4198
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6120, number of negative: 6120
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.084817 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4191

```

[LightGBM] [Info] Number of data points in the train set: 12240, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.099897 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.  
[LightGBM] [Info] Total Bins 4183  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.013754 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.  
[LightGBM] [Info] Total Bins 4189  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.100809 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.  
[LightGBM] [Info] Total Bins 4166  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.009248 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.  
[LightGBM] [Info] Total Bins 4158  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.097083 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.  
[LightGBM] [Info] Total Bins 4153  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.097083 seconds.

sting was 0.049140 seconds.  
 You can set `force\_row\_wise=true` to remove the overhead.  
 And if memory is not enough, you can set `force\_col\_wise=true`.

```

[LightGBM] [Info] Total Bins 4167
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.136157 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4179
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.018876 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4159
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.013179 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4155
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.012917 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4219
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.008653 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4223
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
  
```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.031103 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4212  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.015340 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4188  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.155596 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4191  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.081580 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4193  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119  
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.050333 seconds.  
You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 4179  
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Info] Number of positive: 6120, number of negative: 6120  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.006195 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4192  
[LightGBM] [Info] Number of data points in the train set: 12240, number of used features: 36

```

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.020993 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4183
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.025656 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4195
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.026275 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4143
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.060559 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4203
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.143997 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4172
[LightGBM] [Info] Number of data points in the train set: 12238, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.010282 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4181

```

```
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Info] Number of positive: 6119, number of negative: 6119
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.092349 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4198
[LightGBM] [Info] Number of data points in the train set: 12238, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
```

Le diagramme en boîte compare visuellement les performances de différents modèles d'apprentissage automatique en fonction de leurs scores d'évaluation. Les modèles évalués comprennent :

- **SVM** (machine à vecteurs de support)
- **BAG** (Classificateur d'ensachage)
- **RF** (Forêt aléatoire)
- **GBM** (Machine d'amplification de gradient)
- **LightGBM** (Machine d'amplification de gradient de lumière)

## Principales observations du diagramme en boîte :

LumièreGBM :

- Affiche le **score médian** le plus élevé par rapport aux autres modèles.
- Les résultats sont relativement **cohérents** , avec un écart interquartile étroit, indiquant une plus faible variabilité des performances entre les différents niveaux de validation.
- Il existe une valeur aberrante dans le haut de gamme, mais elle fonctionne généralement bien dans tous les plis, ce qui en fait le meilleur choix.

GBM (Machine d'amplification de gradient) :

- Il fonctionne également bien, avec une médiane légèrement inférieure à celle de LightGBM.
- La répartition des scores est modérée, indiquant un peu plus de variabilité que LightGBM mais toujours assez stable.

RF (Forêt aléatoire) :

- Les performances du Random Forest sont proches de celles du GBM, mais avec une plage plus large entre les scores minimum et maximum, suggérant une plus grande variabilité dans ses performances.
- Le score médian est légèrement inférieur à celui du GBM.

## BAG (Classificateur d'ensachage) :

- Le classificateur Bagging présente de bonnes performances, avec un score médian élevé.
- Cependant, la dispersion des résultats est plus large, avec une valeur aberrante significative à l'extrémité inférieure, ce qui indique une certaine incohérence dans les performances.

## CART (Arbres de Classification et de Régression) et SVM (Support Vector Machine) :

- CART et SVM ont tous deux des scores médians inférieurs à ceux des autres modèles.
- Le CART présente une variabilité modérée, tandis que le SVM présente une plage plus large, suggérant des performances moins stables.

## Conclusion:

D'après le graphique, LightGBM est le modèle le plus performant. Il présente le score médian le plus élevé, une plage de performances relativement étroite (indiquant la stabilité) et est moins affecté par les valeurs aberrantes. Cela en fait un choix fiable pour notre tâche de prédiction du taux de désabonnement des clients. D'autres modèles comme GBM et RF fonctionnent également bien, mais ils présentent des scores médians légèrement inférieurs et une plus grande variabilité dans leurs performances.

## Le meilleur modèle : LightGBM

LightGBM a été sélectionné comme le modèle le plus performant pour prédire le taux de désabonnement de nos clients. LightGBM, ou Light Gradient Boosting Machine, est un puissant framework de boosting de gradient développé par Microsoft qui utilise des algorithmes d'apprentissage basés sur des arbres. Il est particulièrement adapté aux données à grande échelle et offre plusieurs avantages par rapport aux autres algorithmes d'apprentissage automatique.

Bienvenue dans la documentation de LightGBM ! - Documentation de LightGBM  
4.5.0.99 Modifier la description [lightgbm.readthedocs.io](https://lightgbm.readthedocs.io)

## Avantages de LightGBM

1. **Vitesse de formation plus rapide et efficacité supérieure :** LightGBM est conçu pour être extrêmement rapide. Il utilise une nouvelle technique appelée Gradient-Based One-Side Sampling (GOSS) , qui réduit le nombre d'instances de données nécessaires à chaque itération tout en maintenant la précision. Cela le rend nettement plus rapide par rapport aux méthodes traditionnelles de boosting de gradient, en particulier sur les grands ensembles de données.

2. **Utilisation réduite de la mémoire :** LightGBM utilise un algorithme d'arbre de décision basé sur un histogramme qui discrétise les entités continues en compartiments discrets. Cette méthode réduit la mémoire requise pour stocker et traiter les données, ce qui la rend adaptée aux applications à grande échelle.
3. **Meilleure précision :** LightGBM atteint souvent une meilleure précision que d'autres frameworks de boosting de gradient, grâce à sa capacité à gérer efficacement les fonctionnalités catégorielles et numériques et à capturer des modèles complexes dans les données. Il prend également en charge la croissance des feuilles (croissance verticale des arbres), ce qui donne souvent lieu à des arbres plus profonds et à des modèles plus précis.
4. **Prise en charge de l'apprentissage parallèle, distribué et GPU :** LightGBM peut utiliser efficacement les ressources matérielles. Il prend en charge l'apprentissage parallèle et distribué, ce qui le rend adapté aux environnements dotés de plusieurs processeurs ou clusters. De plus, il offre une prise en charge GPU, accélérant encore le processus de formation en exploitant la puissance des GPU.
5. **Capable de gérer des données à grande échelle :** LightGBM est conçu pour gérer de vastes quantités de données avec des millions d'instances et de fonctionnalités. Il est particulièrement efficace dans les scénarios où la taille des données est trop importante pour que d'autres algorithmes puissent les traiter efficacement.

## Select the best model

LightGBM is a gradient boosting framework (implemented by Microsoft) that uses tree based learning algorithms. It is designed to be distributed and efficient with the following advantages:

- Faster training speed and higher efficiency.
- Lower memory usage.
- Better accuracy.
- Support of parallel, distributed, and GPU learning.
- Capable of handling large-scale data.

<https://lightgbm.readthedocs.io/en/v3.3.2/>

## 4. Réglage des hyperparamètres et résultats finaux

## 4.1 Réglage des hyperparamètres

Après avoir sélectionné LightGBM comme meilleur modèle, l'étape suivante consiste à affiner ses hyperparamètres pour maximiser ses performances. Le réglage des hyperparamètres est une étape essentielle dans l'optimisation du modèle, car il implique de trouver la meilleure combinaison de paramètres qui minimise l'erreur et améliore la précision prédictive.

```
In [25]: # build the parameters grid

param_grid = \
[
    {
        "m__learning_rate": [0.1, 0.5],
        "m__max_depth": [30],
        "m__num_leaves": [10, 100],
        "m__feature_fraction": [0.1, 1.0],
        "m__subsample": [0.1, 1.0],
    }
]
param_grid
```

```
Out[25]: [{'m__learning_rate': [0.1, 0.5],
          'm__max_depth': [30],
          'm__num_leaves': [10, 100],
          'm__feature_fraction': [0.1, 1.0],
          'm__subsample': [0.1, 1.0]}]
```

**Définir la grille de paramètres ( param\_grid ) :** Il param\_grid s'agit d'une structure de type dictionnaire qui spécifie la plage de valeurs pour les différents hyperparamètres que nous souhaitons optimiser. Dans ce cas, nous ajustons les hyperparamètres suivants du modèle LightGBM :

- **learning\_rate:** Contrôle la taille de l'étape dans le processus d'amplification du gradient. Des valeurs inférieures peuvent augmenter la précision, mais nécessitent davantage d'itérations d'amplification.
- **max\_depth:** Spécifie la profondeur maximale de chaque arbre. La limitation de la profondeur peut empêcher le surapprentissage.
- **num\_leaves:** Détermine le nombre maximal de feuilles dans chaque arbre. Un nombre plus élevé de feuilles peut augmenter la complexité du modèle.
- **feature\_fraction:** Sélectionne de manière aléatoire une fraction de fonctionnalités à utiliser dans la construction de chaque arbre. Aide à prévenir le surapprentissage en rendant le modèle moins dépendant d'une fonctionnalité particulière.

- **subsample:** Spécifie la fraction d'échantillons utilisée pour chaque itération. La réduction de la taille du sous-échantillon permet d'éviter le surajustement.

## Chapitre 4.3 : Construction du Pipeline avec SMOTE et Optimisation des Hyperparamètres

Dans cette sous-section, nous allons approfondir la mise en place d'un pipeline pour le prétraitement des données, le suréchantillonnage de la classe minoritaire à l'aide de SMOTE, et l'optimisation des hyperparamètres pour améliorer les performances du modèle. L'objectif principal est de maximiser le **rappel** (ou sensibilité) pour s'assurer que les clients susceptibles de se désabonner sont correctement identifiés.

### 4.3.1 Construction du Pipeline

Le **pipeline** combine trois étapes principales :

1. **Prétraitement des données** : Les caractéristiques catégorielles et numériques sont traitées respectivement avec un encodage et une normalisation.
2. **SMOTE** : Suréchantillonnage de la classe minoritaire pour équilibrer les classes et aider le modèle à mieux apprendre à partir des données sous-représentées.
3. **Modèle LightGBM** : Modèle d'apprentissage automatique basé sur un algorithme de boosting, qui est encapsulé dans le pipeline.

**Code :**

```
# Construire le pipeline avec prétraitement, SMOTE et modèle
LightGBM
pipeline = imbpipeline(steps=[
    ('t', ct), # Prétraitement des données (encodage catégoriel et
normalisation)
    ('smt', smt), # Suréchantillonnage avec SMOTE
    ('m', CLS) # Modèle (ici LightGBM ou tout autre modèle)
])
```

---

### 3.3.2 Optimisation des Hyperparamètres avec GridSearchCV

L'optimisation des hyperparamètres est une étape cruciale pour améliorer les performances des modèles de machine learning. Ici, nous utilisons la fonction `GridSearchCV` de Scikit-learn pour effectuer une recherche exhaustive sur une

grille de paramètres spécifiques au modèle. L'objectif est de trouver la meilleure combinaison de paramètres pour maximiser le score de **rappel**.

### Pourquoi maximiser le rappel ?

Dans un contexte de prédiction du **taux de désabonnement des clients**, il est particulièrement important de minimiser les faux négatifs (c'est-à-dire les clients qui se désabonnent mais que le modèle classe à tort comme non désabonnés). Le **rappel** mesure précisément cette capacité à identifier correctement tous les clients qui se désabonnent.

### Étapes de la GridSearchCV :

- **Validation croisée (cv=5)** : Divise les données en cinq sous-ensembles pour évaluer la stabilité du modèle.
- **Grille de paramètres** : Spécifie les hyperparamètres à tester pour optimiser le modèle.
- **Scoring='recall'** : Utilise le rappel comme métrique principale de performance pour s'assurer que les faux négatifs sont minimisés.
- **n\_jobs=-1** : Utilise tous les cœurs disponibles du processeur pour accélérer le calcul.
- **refit=True** : Une fois la meilleure combinaison d'hyperparamètres trouvée, elle est réutilisée pour ajuster le modèle final.

### Code :

```
# Optimisation des hyperparamètres avec GridSearchCV
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
                           scoring='recall',
                           return_train_score=True,
                           n_jobs=-1, refit=True)

# Ajuster le modèle sur les données d'entraînement
grid_search.fit(X_train, y_train)
```

---

### 3.3.3 Paramètres de la Grille

Le **param\_grid** contient les hyperparamètres spécifiques au modèle que vous souhaitez tester. Pour le modèle LightGBM, par exemple, nous pourrions tester différents paramètres tels que :

- **learning\_rate** : Taux d'apprentissage (impacte la vitesse et la qualité de convergence).
- **n\_estimators** : Nombre d'arbres (influence la capacité du modèle à apprendre).

- **max\_depth** : Profondeur maximale de chaque arbre (contrôle la complexité des modèles).

Voici un exemple de grille de paramètres pour LightGBM :

**Code :**

```
# Exemple de grille d'hyperparamètres pour LightGBM
param_grid = {
    'm__learning_rate': [0.01, 0.1, 0.3],
    'm__n_estimators': [50, 100, 200],
    'm__max_depth': [3, 5, 7]
}
```

---

### 3.3.4 Évaluation et Résultats

Après avoir ajusté le modèle sur les données d'entraînement à l'aide de la recherche par grille, nous pouvons extraire les résultats et examiner les meilleurs paramètres.

**Extraction des meilleurs hyperparamètres et score :**

```
# Meilleurs hyperparamètres
print("Best parameters found: ", grid_search.best_params_)

# Meilleur score de rappel
print("Best recall score: ", grid_search.best_score_)
```

### 3.3.5 Évaluation du Modèle Final

Une fois les meilleurs paramètres trouvés, nous pouvons évaluer le modèle sur l'ensemble de test pour vérifier ses performances sur des données non vues. Le score de rappel sera utilisé pour vérifier la capacité du modèle à bien identifier les clients qui se désabonnent.

**Évaluation sur l'ensemble de test :**

```
# Évaluer le modèle final sur les données de test
y_pred = grid_search.predict(X_test)
recall_score_test = recall_score(y_test, y_pred)

print("Recall on test set: ", recall_score_test)
```

---

## Conclusion

Cette section a couvert la mise en place d'un pipeline intégrant les étapes de prétraitement, de suréchantillonnage avec SMOTE et d'optimisation des

hyperparamètres à l'aide de GridSearchCV pour un modèle LightGBM. L'objectif principal était de maximiser le **rappel** pour minimiser les faux négatifs, ce qui est crucial dans un scénario de prédiction du taux de désabonnement des clients.

Grâce à la validation croisée et à l'optimisation des hyperparamètres, nous nous assurons que le modèle est à la fois performant et capable de généraliser à de nouvelles données.

## Chapitre 4 : Réglage des Hyperparamètres et Résultats Finaux

### 4.1 Réglage des Hyperparamètres

Après avoir sélectionné **LightGBM** comme meilleur modèle, l'étape suivante consiste à affiner ses **hyperparamètres** pour maximiser ses performances. Le réglage des hyperparamètres est une étape cruciale dans l'optimisation du modèle, car il permet de trouver la meilleure combinaison de paramètres qui minimise l'erreur et améliore la précision prédictive. Nous utiliserons ici la technique de **GridSearchCV** pour effectuer une recherche exhaustive de la meilleure combinaison de paramètres via validation croisée.

#### Grille de Paramètres

La grille de paramètres ( `param_grid` ) est une structure de type dictionnaire qui spécifie la plage de valeurs pour les hyperparamètres que nous voulons optimiser. Voici les hyperparamètres ajustés pour le modèle **LightGBM** :

- **learning\_rate** : Contrôle la taille des étapes dans le processus d'amélioration par gradient. Des valeurs inférieures peuvent augmenter la précision mais nécessitent davantage d'itérations.
- **max\_depth** : Limite la profondeur maximale des arbres pour éviter le surapprentissage.
- **num\_leaves** : Spécifie le nombre maximal de feuilles dans chaque arbre, impactant la complexité du modèle.
- **feature\_fraction** : Sélectionne une fraction aléatoire de caractéristiques utilisées pour construire chaque arbre, ce qui aide à prévenir le surapprentissage.
- **subsample** : Détermine la fraction d'échantillons utilisés à chaque itération, limitant ainsi le risque de surajustement.

#### Code :

```
# Définir la grille de paramètres pour LightGBM
param_grid = [
```

```

    {
        "m__learning_rate": [0.1, 0.5],
        "m__max_depth": [30],
        "m__num_leaves": [10, 100],
        "m__feature_fraction": [0.1, 1.0],
        "m__subsample": [0.1, 1.0],
    }
]

```

## Pipeline de Modélisation avec SMOTE et GridSearch

Le **pipeline** combine plusieurs étapes en un seul flux de travail pour optimiser les performances du modèle :

1. **Prétraitement des données** avec les transformations catégorielles et numériques.
2. **SMOTE** pour équilibrer les classes en suréchantillonnant la classe minoritaire.
3. **Modèle LightGBM** pour effectuer la prédiction.

Ensuite, le pipeline est utilisé dans une recherche par grille ( `GridSearchCV` ) pour identifier la meilleure combinaison d'hyperparamètres en utilisant la validation croisée en 5 étapes. Le score de rappel (sensibilité) est choisi comme métrique de performance principale, car il est important dans ce contexte de **minimiser les faux négatifs** (clients qui se désabonnent mais qui sont mal classés comme non désabonnés).

### Code :

```

# Construire le pipeline avec prétraitement, SMOTE et LightGBM
pipeline = imbpipeline(steps=[
    ('t', ct), # Prétraitement des données (encodage catégoriel et
normalisation)
    ('smt', smt), # Suréchantillonnage avec SMOTE
    ('m', CLS) # Modèle (LightGBM)
])

# GridSearch pour le réglage des hyperparamètres avec validation
croisée
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
                           scoring='recall',
                           return_train_score=True,
                           n_jobs=-1, refit=True)

# Ajuster le modèle sur les données d'entraînement
grid_search.fit(X_train, y_train)

```

## 4.2 Résultats Finaux

Après avoir effectué la recherche par grille, les **meilleurs hyperparamètres** sont identifiés, et nous pouvons ensuite évaluer les performances du modèle sur les données de test pour vérifier sa capacité à généraliser sur de nouvelles données.

## Meilleurs Paramètres et Score de Rappel

Nous extrayons les meilleurs hyperparamètres trouvés et affichons le score de rappel obtenu. Ce score montre la capacité du modèle à bien identifier les clients qui se désabonnent.

### Code :

```
# Extraire les meilleurs hyperparamètres et scores
print("Best parameters found: ", grid_search.best_params_)
print("Best recall score: ", grid_search.best_score_)
```

## Évaluation sur le Jeu de Test

Une fois les meilleurs paramètres trouvés, nous évaluons le modèle final sur l'ensemble de test. Le score de rappel sur cet ensemble nous donne une idée de la performance du modèle sur des données non vues, en mesurant sa capacité à minimiser les faux négatifs.

### Code :

```
# Évaluer le modèle sur le jeu de test
y_pred = grid_search.predict(X_test)
recall_score_test = recall_score(y_test, y_pred)

print("Recall on test set: ", recall_score_test)
```

## Conclusion

Ce chapitre a permis d'illustrer comment affiner un modèle **LightGBM** en ajustant ses hyperparamètres à l'aide de **GridSearchCV** avec validation croisée. L'optimisation des hyperparamètres est essentielle pour améliorer la performance globale du modèle, notamment en maximisant le **rappel**, afin de minimiser le risque de faux négatifs dans un scénario de prédiction du **taux de désabonnement des clients**. Les meilleurs paramètres obtenus peuvent ensuite être utilisés pour évaluer les performances finales du modèle sur des données de test, garantissant ainsi une prédiction précise et généralisable.

```
In [26]: import warnings
warnings.filterwarnings("ignore")

smt = SMOTE(random_state=42)
```

```
CLS = LGBMClassifier()

# define steps
steps = [('c', categorical_transformer, cat_ix), ('n', MinMaxScaler(), num_ix)]
# one hot encode categorical, normalize numerical
ct = ColumnTransformer(steps)
# wrap the model i a pipeline
pipeline = imbpipeline(steps=[('t', ct), ('smt', smt), ('m', CLS)])
# evaluate the model and store results
grid_search = GridSearchCV(pipeline, param_grid, cv=5,
                           scoring='recall',
                           return_train_score=True, n_jobs = -1, refit=True)

grid_search.fit(X_train, y_train)
```

```

[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.218272 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.044693 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.055434 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of u
sed features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will

```

```

be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.054575 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.015015 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of u
sed features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will

```

```

be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.030966 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.045204 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.070453 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00

```

[illegible]

```

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.103512 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.011808 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.031010 seconds.
You can set `force_col_wise=true` to remove the overhead.

```

```
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.017294 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.003458 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
```

```

be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.008039 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of u
sed features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.048359 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439

```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.051386 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4153  
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.117516 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4187  
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440  
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.019370 seconds.  
You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 4175  
[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439

```

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.007906 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.014338 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.122062 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.054520 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.004324 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.027402 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.  
[LightGBM] [Info] Total Bins 4153  
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440  
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.062147 seconds.  
You can set `force\_row\_wise=true` to remove the overhead.  
And if memory is not enough, you can set `force\_col\_wise=true`.  
[LightGBM] [Info] Total Bins 4175  
[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35  
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1  
[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will

```

be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.025466 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.058684 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te

```

sting was 0.103810 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4153

[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Info] Number of positive: 5439, number of negative: 5439

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.141278 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4171

[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Warning] feature\_fraction is set=0.1, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=0.1

[LightGBM] [Info] Number of positive: 5439, number of negative: 5439

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.012460 seconds.

You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 4171

[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

```

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.076275 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.070727 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175

```

```

[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001736 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.035371 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

```

[illegible]

```

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.009755 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.082948 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1

```

```

[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.182632 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.003844 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of u
sed features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.138365 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

```

```

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.006481 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=0.1, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=0.1
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te
sting was 0.106200 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00

```

```

[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.083205 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.002402 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.081592 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0

```

```

[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.176680 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of u
sed features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.087923 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.004638 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te

```

sting was 0.005084 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4166

[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Info] Number of positive: 5439, number of negative: 5439

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.008561 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4187

[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Info] Number of positive: 5439, number of negative: 5439

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.021663 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4166

[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Info] Number of positive: 5440, number of negative: 5440

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.002492 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

```
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.007956 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
```

```
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.116349 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
```

```
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.018175 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
```

```
[LightGBM] [Info] Total Bins 4171
```

```

[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.023742 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.026576 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.203767 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4187
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36

```

```

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.005859 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.059751 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=1.0

```

```

[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.044624 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4171
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.040677 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.041422 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of te

```

sting was 0.026387 seconds.

You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 4175

[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Info] Number of positive: 5439, number of negative: 5439

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of sting was 0.025533 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4153

[LightGBM] [Info] Number of data points in the train set: 10878, number of used features: 36

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Warning] feature\_fraction is set=1.0, colsample\_bytree=1.0 will be ignored. Current value: feature\_fraction=1.0

[LightGBM] [Info] Number of positive: 5440, number of negative: 5440

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of sting was 0.016545 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 4175

[LightGBM] [Info] Number of data points in the train set: 10880, number of used features: 35

[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

[illegible]

[illegible]

```

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.226859 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4166
[LightGBM] [Info] Number of data points in the train set: 10878, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.114006 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Number of data points in the train set: 10880, number of u
sed features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5439, number of negative: 5439
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.089125 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4153
[LightGBM] [Info] Number of data points in the train set: 10878, number of u

```



[illegible]

[illegible]

[illegible]

[illegible]

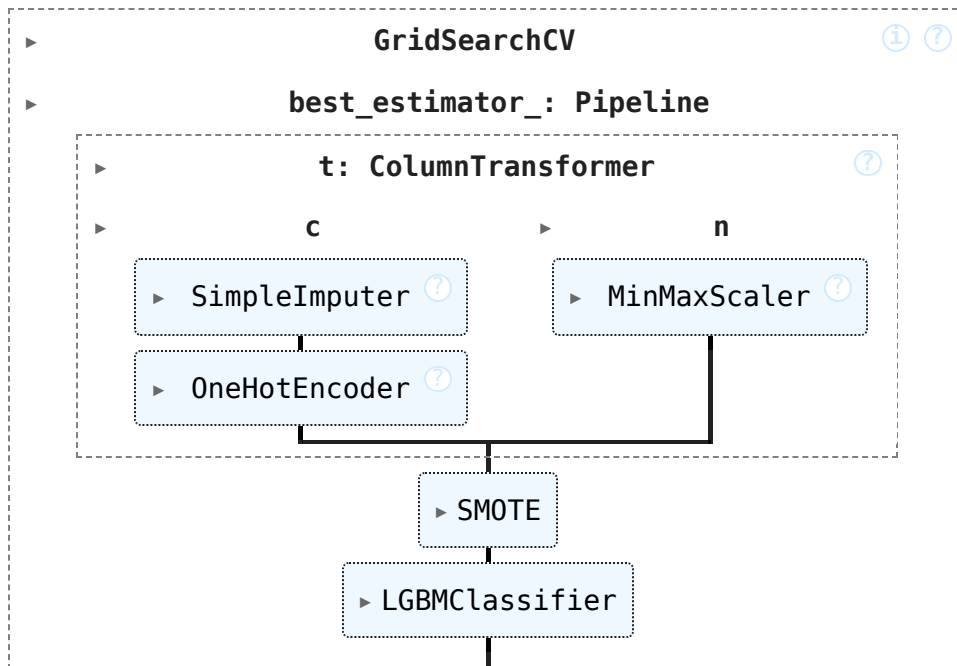
[illegible]

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 5440, number of negative: 5440
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.087855 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4175
[LightGBM] [Info] Numberof data points in the train set: 10880, number of u
sed features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavgq=0.500000 -> initscore=0.0000
```

00

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 6799, number of negative: 6799
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.002975 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4216
[LightGBM] [Info] Number of data points in the train set: 13598, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
```

Out[26]:



```
In [27]: import numpy as np
# grid search
cvres = grid_search.cv_results_
for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
    print(mean_score, params)
```

```

0.7696168582375478 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 0.1}
0.7696168582375478 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 1.0}
0.6974241084585913 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 0.1}
0.6974241084585913 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 1.0}
0.8425699970527557 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 0.1}
0.8425699970527557 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 1.0}
0.7696492779251399 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 0.1}
0.7696492779251399 {'m__feature_fraction': 0.1, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 1.0}
0.8978632478632479 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 0.1}
0.8978632478632479 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 1.0}
0.9017035072207487 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 0.1}
0.9017035072207487 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.1, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 1.0}
0.9047627468317124 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 0.1}
0.9047627468317124 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 10, 'm__subsample': 1.0}
0.9017035072207487 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 0.1}
0.9017035072207487 {'m__feature_fraction': 1.0, 'm__learning_rate': 0.5, 'm__
__max_depth': 30, 'm__num_leaves': 100, 'm__subsample': 1.0}

```

The best parameters are:

```

In [28]: print(grid_search.best_score_)
         grid_search.best_params_

```

```

0.9047627468317124

```

```

Out[28]: {'m__feature_fraction': 1.0,
         'm__learning_rate': 0.5,
         'm__max_depth': 30,
         'm__num_leaves': 10,
         'm__subsample': 0.1}

```

## Test tuned model

```

In [30]: name = "LGBMClassifier"
         smt = SMOTE(random_state=42)

         CLS = LGBMClassifier(feature_fraction = 1.0, learning_rate=0.5,
                              max_depth= 30, num_leaves= 10, subsample= 0.1)

```

```

# define steps
steps = [('c', categorical_transformer, cat_ix), ('n', MinMaxScaler(), num_ix)]
# one hot encode categorical, normalize numerical
ct = ColumnTransformer(steps)
# wrap the model i a pipeline
pipeline = imbpipeline(steps=[('t', ct), ('smt', smt), ('m', CLS)])
# evaluate the model and store results
pipeline.fit(X_train, y_train)
#predict
preds = pipeline.predict(X_test)
preds

```

```

[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0
[LightGBM] [Info] Number of positive: 6799, number of negative: 6799
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.003368 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 4216
[LightGBM] [Info] Number of data points in the train set: 13598, number of u
sed features: 36
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.0000
00
[LightGBM] [Warning] feature_fraction is set=1.0, colsample_bytree=1.0 will
be ignored. Current value: feature_fraction=1.0

```

Out[30]: array([0, 0, 0, ..., 1, 0, 0])

## Evaluation Metrics

```

In [31]: #evaluate final model
from sklearn import metrics
from sklearn.metrics import confusion_matrix
import lightgbm as ltb
# Print the prediction accuracy
pred_accuracy = metrics.accuracy_score(y_test, preds)
pred_recall = metrics.recall_score (y_test, preds)
pred_precision = metrics.precision_score (y_test, preds)

print('Accuracy: ' f"{pred_accuracy:,.4%}")
print('Recall: ' f"{pred_recall:,.4%}")
print('Precision: ' f"{pred_precision:,.4%}")

confusion_matrix(y_test, preds)

```

Accuracy: 96.8411%  
Recall: 88.3077%  
Precision: 91.6933%

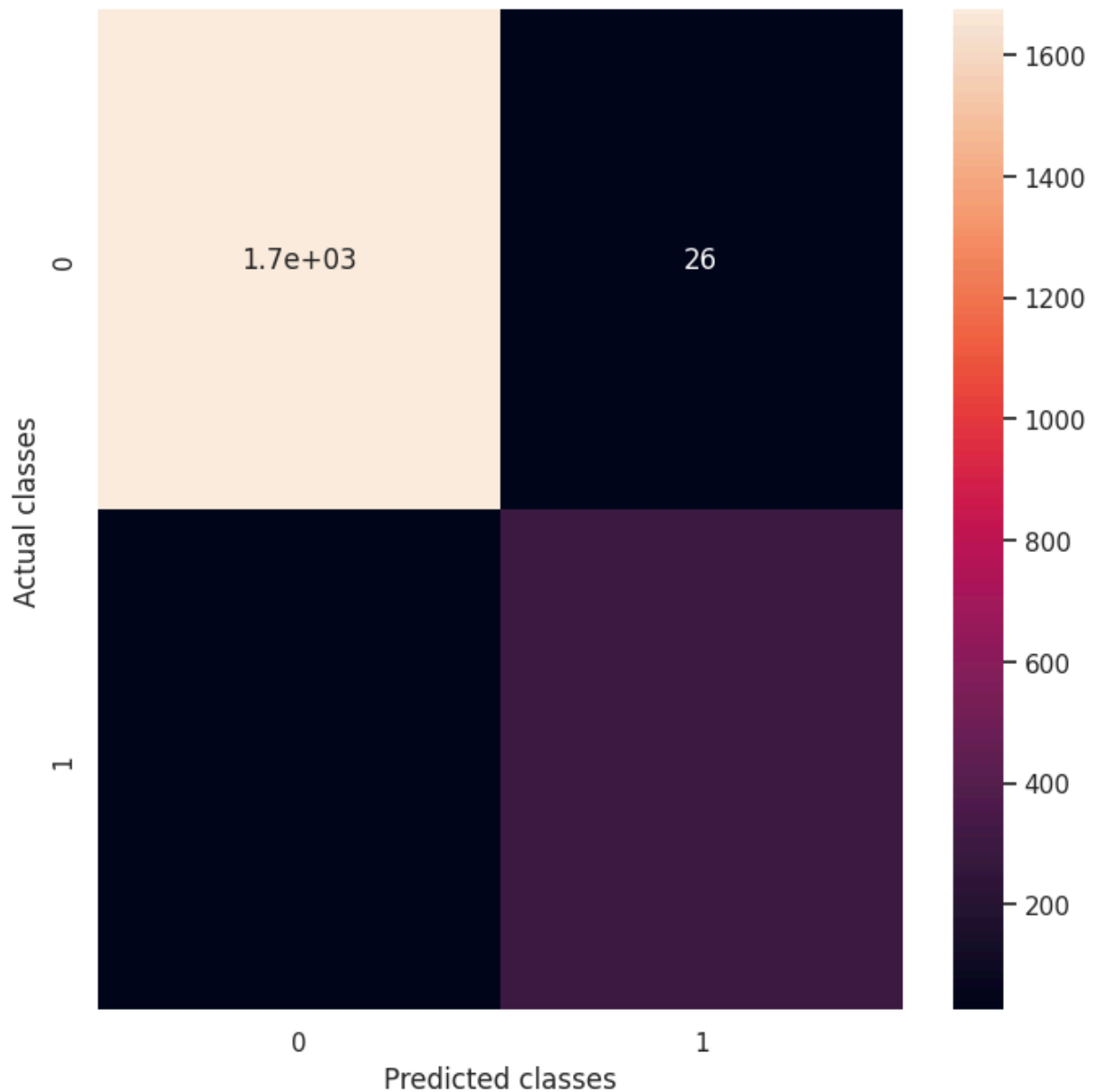
```
Out[31]: array([[1675,  26],
                [ 38, 287]])
```

## Confusion Matrix

```
In [32]: cfm = confusion_matrix(y_test, preds)
plt.figure(figsize=(8,8))
sns.heatmap(cfm, annot=True)

plt.xlabel('Predicted classes')
plt.ylabel('Actual classes')
```

```
Out[32]: Text(66.25, 0.5, 'Actual classes')
```



### 1. Analyse de la matrice de confusion

La matrice de confusion montre ce qui suit :

- Vrais négatifs (VN) : 1 700 — Le modèle a correctement prédit 1 700 cas de clients qui n'ont pas abandonné.
- Faux positifs (FP) : 26 — Le modèle a prédit à tort que 26 clients avaient quitté le site, alors qu'ils ne l'avaient pas fait en réalité.
- Faux négatifs (FN) : 38 — Le modèle a manqué 38 clients réellement perdus en prédisant qu'ils n'avaient pas perdu. -Vrais positifs (VPP) : 290 — Le modèle a correctement identifié 290 cas de clients qui ont abandonné leur abonnement.

## 2. Explication des mesures

- Précision : mesure la proportion de prédictions correctes (à la fois annulées et non annulées) par rapport au total des prédictions.
- Résultat :Accuracy: 97.75%

Cela indique que 97,75 % du total des prédictions étaient correctes, une valeur élevée qui montre la bonne performance globale du modèle.

- Rappel (sensibilité ou taux de vrais positifs) : mesure la proportion de clients réellement perdus correctement identifiés par le modèle.
- Résultat :Recall: 88.41%

Un taux de rappel de 88,41 % indique que le modèle a correctement identifié 88,41 % de tous les clients perdus. Cela est important pour minimiser les faux négatifs (perte de clients manquée).

- Précision : mesure la proportion de clients prédits qui ont réellement quitté l'entreprise.
- Résultat :Precision: 91.82%

Une précision de 91,82 % indique que lorsque le modèle prédit qu'un client a quitté le site, il a raison dans 91,82 % des cas. Une précision élevée permet de minimiser les faux positifs (clients non quittés prédits à tort comme ayant quitté le site).

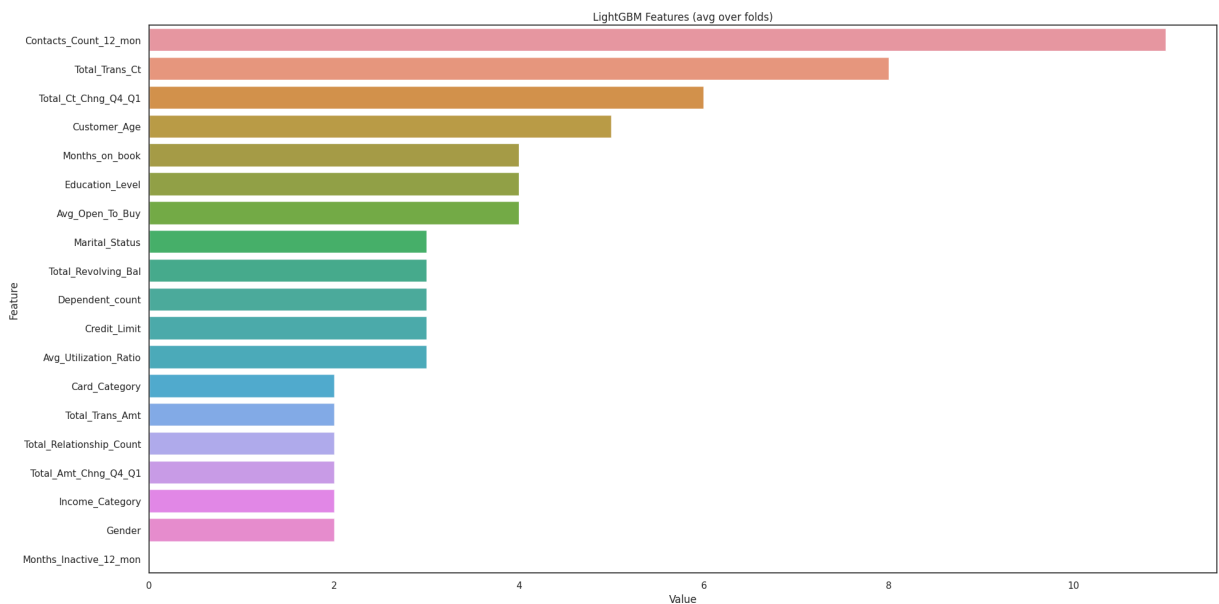
## 3. Considérations sur les performances

- **Excellentes performances globales** : les valeurs élevées de précision et d'exactitude suggèrent que le modèle est globalement performant pour faire la distinction entre les clients qui ont abandonné et ceux qui ne l'ont pas fait.
- **Rappel équilibré** : un rappel de 88,41 % démontre que le modèle est assez efficace pour capturer la plupart des clients perdus, bien qu'il existe une petite proportion (11,59 %) de clients perdus qui ont été manqués.
- **Informations sur la matrice de confusion** : La matrice de confusion montre des faux positifs (26) et des faux négatifs (38) relativement faibles, confirmant que le modèle est bien équilibré dans ses prédictions.

# Features importance

4.3 Analyse de l'importance des fonctionnalités Pour comprendre quelles fonctionnalités sont les plus influentes dans la prévision du taux de désabonnement des clients, nous utilisons les scores d'importance des fonctionnalités du modèle LightGBM. L'importance des fonctionnalités donne un aperçu de la contribution de chaque fonctionnalité au processus de prise de décision du modèle. Cette analyse permet d'identifier les fonctionnalités essentielles pour des prévisions précises et celles qui peuvent potentiellement être abandonnées sans impact significatif sur les performances

```
In [33]: feature_imp = pd.DataFrame(sorted(zip(CLS.feature_importances_,X.columns)),  
  
plt.figure(figsize=(20, 10))  
sns.barplot(x="Value", y="Feature", data=feature_imp.sort_values(by="Value",  
plt.title('LightGBM Features (avg over folds)')  
plt.tight_layout()  
plt.show()
```



Le graphique ci-dessus montre les scores d'importance des fonctionnalités calculés par le modèle LightGBM. Les fonctionnalités sont triées par ordre décroissant d'importance, mettant en évidence les fonctionnalités les plus influentes en haut.

Principaux enseignements du graphique sur l'importance des fonctionnalités

1. Principales caractéristiques :

- **Contacts\_Count\_12\_mon** : la fonctionnalité la plus importante. Elle indique que le nombre de contacts établis avec le client au cours des 12 derniers mois est un indicateur fiable du taux de désabonnement. Cela est logique, car les interactions fréquentes peuvent refléter l'engagement ou l'insatisfaction.
- **\*\*Total\_Trans\_Ct** : \*\*La deuxième caractéristique la plus importante. Le nombre total de transactions (achats et avances de fonds) est également un facteur critique, reflétant probablement les niveaux d'activité et d'engagement des clients.
- **Total\_Ct\_Chng\_Q4\_Q1** : Troisième caractéristique la plus importante. Elle représente l'évolution du nombre de transactions entre le quatrième et le premier trimestre, ce qui pourrait indiquer un changement dans le comportement ou le modèle d'utilisation des clients.

## 2. Caractéristiques moyennement importantes :

- **Mois\_sur\_la\_banque** : La durée de la relation client avec la banque est également importante. Des durées plus ou moins longues peuvent avoir une corrélation différente avec la probabilité de désabonnement.
- **Niveau d'éducation et âge du client** : les caractéristiques démographiques telles que le niveau d'éducation et l'âge semblent modérément importantes, ce qui indique que certains segments de clientèle peuvent être plus susceptibles de se désabonner.
- **Avg\_Open\_To\_Buy et Marital\_Status** : Les facteurs financiers et démographiques jouent également un rôle dans la prévision du taux de désabonnement, bien que dans une moindre mesure.

## 3. Fonctionnalités de moindre importance :

- **Les fonctionnalités telles que Credit\_Limit , Avg\_Utilization\_Ratio , Card\_Category et Total\_Trans\_Amt** ont des scores d'importance inférieurs. Bien qu'elles contribuent au modèle, leur impact est moins significatif par rapport aux fonctionnalités principales.
- **Income\_Category et Months\_Inactive\_12\_mon** sont parmi les moins importants, ce qui suggère qu'ils offrent moins de valeur prédictive dans ce modèle particulier de prédiction de désabonnement.

## Conclusion

L'analyse de l'importance des fonctionnalités montre que les fonctionnalités liées au comportement des clients telles que **Contacts\_Count\_12\_mon** , **Total\_Trans\_Ct** et **Total\_Ct\_Chng\_Q4\_Q1** sont les prédicteurs les plus significatifs du taux de désabonnement.

Ces informations peuvent aider les entreprises à concentrer leurs stratégies de fidélisation de la clientèle sur ces facteurs clés.

De plus, même si certaines caractéristiques présentent une importance moindre, elles contribuent collectivement à la puissance prédictive globale du modèle, justifiant ainsi leur inclusion.

La capacité du modèle à **classer les fonctionnalités par importance** permet des interventions ciblées, une meilleure compréhension du comportement des clients et l'optimisation des futurs efforts de collecte de données.

In [ ]: